B.TECH CSE | JAVA PROGRAMMING

Java Programming Study Notes

Clear concepts, code examples, and exam practice

Module-wise notes based on the syllabus supplied for these notes

Syllabus version: W.e.f. session 2023-2024

Simple explanation first. Formal definition next. Then a code trace.

STUDENT EDITION | 174 PAGES | 3 MODULES

A clear route through Java

How to study each topic

- Read the simple explanation before memorizing the formal definition.
- Trace code by writing variable values, reference targets, and output one step at a time.
- For classes, label the class, object, reference, fields, constructor, and method call.
- For inheritance/exceptions/threads, draw the parent chain, call stack, or thread-state path.
- For GUI/JDBC, follow the event or connection path and note APIs that depend on a legacy/runtime context.

Code version and syllabus context

- Core language examples use familiar Java syntax suitable for introductory courses.
- Applet and finalize() are retained because they appear in your syllabus; their current Java status is clearly explained in those topics.
- GUI and JDBC code uses standard API patterns; a database connection needs a matching driver and configuration.

QUICK RECALL

Before reading the output, predict it yourself and explain why each line runs.

MODULE-I - what to study and in what order

Module goal

Build Java's execution model and type rules, then practice operators/control flow and object-oriented programming.

Syllabus map - exact topic order

- Java Language: History, Lineage, and Creation of Java; Java's Magic Code: The Byte Code; Java Class File Format; Java's Buzzwords; Evolution of Java.
- Object Orientation concepts: Object-Oriented Concepts; Class, Object, and Their Significance.
- Environment Variables: Environment Variables and Java Setup.
- Data Types, Variables and Array: Java as a Strongly Typed Language; Primitive Types; Non-Primitive Types; Wrapper Classes; Variables: Scope and Lifetime; Arrays: Declaration and Basic Use; Type Conversion and Casting; Automatic Type Promotions.
- Operators: Arithmetic Operators; Bitwise Operators; Relational Operators; Assignment Operators; The ? Operator (Conditional Operator); Operator Precedence.
- Control Statements: Selection Statements; Iteration Statements; Jump Statements.
- Introducing Classes: Class Fundamentals; Creating and Operating Objects; Methods and Defining Methods; Argument Passing Mechanism; Method Overloading; Recursion; Nested and Inner Classes; Anonymous Classes; Abstract Classes and Interfaces; Garbage Collection and Object Reachability; finalize() Method.

Recommended study order

- Start with Java history, bytecode, class files, and its design goals.
- Study types, variables, arrays, operators, and control statements.
- Finish with classes, methods, references, inheritance of behavior, and object cleanup.

QUICK RECALL

Explain how a .java file becomes executable behavior on a JVM.

TOPIC 01 / 75 | Java Language

CONCEPT AND DEFINITIONS

History, Lineage, and Creation of Java

Simple definition

Java began as a language and platform project for small, connected devices, then grew into a general-purpose platform for desktop, server, and enterprise software.

Formal / technical definition

Java originated in Sun Microsystems' Green Project in the early 1990s. James Gosling and the team developed Oak for embedded/consumer-electronic goals; the language was renamed Java and publicly introduced in 1995, with JDK 1.0 released in 1996. Its syntax drew familiarity from C/C++ while its runtime and safety model were separately designed.

Notation and how to read it

1991 Green Project; 1992 Oak work; 1995 Java introduced; 1996 JDK 1.0

How it works

- Identify the compiler, class-file, runtime, or release-history stage being discussed.
- Explain the role it plays in portability, compatibility, or execution.
- Separate a design goal from a guarantee and use the correct Java terminology.

Quick map / trace / syntax

```
Need for portable software -> Oak/Java language + libraries + VM -> multiple computing environments
```

QUICK RECALL

Name the original Oak project, the rename to Java, the compiler/VM model, and one reason it mattered.

TOPIC 01 / 75 | Java Language

CODE EXAMPLE AND EXAM HELP

History, Lineage, and Creation of Java

Worked code example and output

```
History sketch: Green Project -> Oak -> Java -> source compiled to portable class files ->
JVM implementations for different platforms.
```

Useful distinction / limitation

Java is influenced by C/C++ syntax but is not a direct superset of C++; it removes or changes several low-level features.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Name the original Oak project, the rename to Java, the compiler/VM model, and one reason it mattered.
- Likely follow-up: Java is influenced by C/C++ syntax but is not a direct superset of C++; it removes or changes several low-level features.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Name the original Oak project, the rename to Java, the compiler/VM model, and one reason it mattered.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 02 / 75 | Java Language CONCEPT AND DEFINITIONS

Java's Magic Code: The Byte Code

Simple definition

Java source is compiled into bytecode, an intermediate instruction format that a Java Virtual Machine can run on different systems.

Formal / technical definition

The Java compiler translates source constructs into JVM instructions stored in class files. A JVM implementation loads, verifies, links, and executes supported class-file versions, interpreting or compiling instructions to native machine code.

Notation and how to read it

source (.java) --javac--> bytecode (.class) --JVM--> machine-specific execution

How it works

- Identify the compiler, class-file, runtime, or release-history stage being discussed.
- Explain the role it plays in portability, compatibility, or execution.
- Separate a design goal from a guarantee and use the correct Java terminology.

Quick map / trace / syntax

```
Same class file + compatible JVMs -> platform portability
JVM may interpret, JIT-compile, or mix both
```

QUICK RECALL

Draw the source-compile-class-file-JVM path and distinguish bytecode from native code.

TOPIC 02 / 75 | Java Language

CODE EXAMPLE AND EXAM HELP

Java's Magic Code: The Byte Code

Worked code example and output

```
Hello.java -> javac Hello.java -> Hello.class (bytecode) -> java Hello -> JVM on this operating system
```

Useful distinction / limitation

Bytecode is not ordinary CPU machine code, and portability still depends on compatible JVMs, APIs, and environment assumptions.

Common mistakes to avoid

- Bytecode is JVM instruction format, not native CPU machine code.
- Portability still requires compatible JVMs/APIs and environment assumptions.

Quick viva and follow-up cues

- Core answer: Draw the source-compile-class-file-JVM path and distinguish bytecode from native code.
- Likely follow-up: Bytecode is not ordinary CPU machine code, and portability still depends on compatible JVMs, APIs, and environment assumptions.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Draw the source-compile-class-file-JVM path and distinguish bytecode from native code.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 03 / 75 | Java Language CONCEPT AND DEFINITIONS

Java Class File Format

Simple definition

A .class file packages bytecode with information the JVM needs to load a class, check it, and connect its methods and fields.

Formal / technical definition

A class file is a specified binary structure containing a magic number, version fields, constant pool, access flags, class and superclass indexes, interfaces, field/method descriptions, and attributes. The first four bytes are the magic value 0xCAFEBABE.

Notation and how to read it

ClassFile = magic + minor/major version + constant pool + flags/hierarchy + fields + methods + attributes

How it works

- Identify the compiler, class-file, runtime, or release-history stage being discussed.
- Explain the role it plays in portability, compatibility, or execution.
- Separate a design goal from a guarantee and use the correct Java terminology.

Quick map / trace / syntax

```
JVM checks class-file structure/version -> resolves symbolic references -> verifies bytecode -> executes methods
```

QUICK RECALL

State the magic number and list the major structural sections in order.

TOPIC 03 / 75 | Java Language

CODE EXAMPLE AND EXAM HELP

Java Class File Format

Worked code example and output

```
Java source class Hello { ... } -> Hello.class
Header begins CAFEBABE; later data identifies version, constants, fields, methods, and attributes.
```

Useful distinction / limitation

The .class file is not just a text file with instructions; it has a structured, versioned binary format.

Common mistakes to avoid

- A .class file is structured binary data, not a text listing.
- The magic value is 0xCAFEBABE; version compatibility also matters.

Quick viva and follow-up cues

- Core answer: State the magic number and list the major structural sections in order.
- Likely follow-up: The .class file is not just a text file with instructions; it has a structured, versioned binary format.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: State the magic number and list the major structural sections in order.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 04 / 75 | Java Language

CONCEPT AND DEFINITIONS

Java's Buzzwords

Simple definition

Java's well-known design words describe goals such as portability, safety, object-oriented structure, and support for concurrent programs.

Formal / technical definition

Commonly cited Java characteristics include simple/familiar, object-oriented, distributed/network-oriented, robust, secure, architecture-neutral, portable, high-performance, interpreted/JIT-compiled, multithreaded, and dynamic. These are design descriptions, not guarantees without limits.

Notation and how to read it

Buzzwords: simple, object-oriented, distributed, robust, secure, architecture-neutral, portable, high-performance, interpreted/JIT, multithreaded, dynamic

How it works

- Identify the compiler, class-file, runtime, or release-history stage being discussed.
- Explain the role it plays in portability, compatibility, or execution.
- Separate a design goal from a guarantee and use the correct Java terminology.

Quick map / trace / syntax

```
Language + standard libraries + class-file format + JVM contribute together to many platform characteristics
```

QUICK RECALL

For each buzzword, give the design mechanism and one limitation.

TOPIC 04 / 75 | Java Language

CODE EXAMPLE AND EXAM HELP

Java's Buzzwords

Worked code example and output

```
Architecture-neutral: bytecode targets the JVM. Robust: type checks, exceptions, and memory
management help catch classes of errors. Multithreaded: the platform has thread APIs.
```

Useful distinction / limitation

Do not claim Java is perfectly secure, always faster than native code, or entirely free of unsafe/native interfaces.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: For each buzzword, give the design mechanism and one limitation.
- Likely follow-up: Do not claim Java is perfectly secure, always faster than native code, or entirely free of unsafe/native interfaces.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: For each buzzword, give the design mechanism and one limitation.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 05 / 75 | Java Language

CONCEPT AND DEFINITIONS

Evolution of Java

Simple definition

Java has changed over time while keeping a strong focus on compatibility; major releases added language features and platform APIs.

Formal / technical definition

Java evolves through Java SE releases, specification changes, APIs, and JDK tools. Important language milestones include Java 5 (generics, enums, annotations, enhanced for, autoboxing), Java 8 (lambdas and default methods), and Java 9 (modules); later releases continue regular feature updates.

Notation and how to read it

Evolution = language features + libraries + JVM/class-file updates + tools; compatibility is managed across releases

How it works

- Identify the compiler, class-file, runtime, or release-history stage being discussed.
- Explain the role it plays in portability, compatibility, or execution.
- Separate a design goal from a guarantee and use the correct Java terminology.

Quick map / trace / syntax

```
Older Java program -> newer JVM often supported, but removed/deprecated APIs and changed
defaults still need migration checks
```

QUICK RECALL

Use the major feature milestones from the syllabus first; label current release facts with the date checked.

TOPIC 05 / 75 | Java Language

CODE EXAMPLE AND EXAM HELP

Evolution of Java

Worked code example and output

```
Milestone line: JDK 1.0 (1996) -> Java 5 (2004) -> Java 8 (2014) -> Java 9 (2017) -> Java 17 (2021) -> Java 21 (2023) -> Java 25 (2025) -> Java 27 (2026).
```

Useful distinction / limitation

Release numbers are time-based in modern Java; do not treat every release as an LTS release.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Use the major feature milestones from the syllabus first; label current release facts with the date checked.
- Likely follow-up: Release numbers are time-based in modern Java; do not treat every release as an LTS release.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Use the major feature milestones from the syllabus first; label current release facts with the date checked.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 06 / 75 | Object Orientation concepts

CONCEPT AND DEFINITIONS

Object-Oriented Concepts

Simple definition

Object-oriented programming organizes a program around objects that combine data with operations on that data.

Formal / technical definition

Core OOP ideas commonly include abstraction (expose essentials), encapsulation (protect/manage state through an interface), inheritance (derive related types), and polymorphism (one interface, multiple implementations). Java expresses these with classes, objects, interfaces, and access control.

Notation and how to read it

class -> blueprint; object -> instance; field -> state; method -> behavior; interface -> contract

How it works

- Identify the class/object and its state/behavior.
- Apply abstraction, encapsulation, inheritance, or polymorphism to the example.
- State one benefit and one design limitation.

Quick map / trace / syntax

```
Client -> public method -> validated state change
private fields remain inside the class boundary
```

QUICK RECALL

Define abstraction, encapsulation, inheritance, and polymorphism with short Java examples.

TOPIC 06 / 75 | Object Orientation concepts

CODE EXAMPLE AND EXAM HELP

Object-Oriented Concepts

Worked code example and output

```
A BankAccount object owns balance data and exposes deposit(amount); callers use the
operation instead of changing internal state arbitrarily.
```

Useful distinction / limitation

Inheritance is not the only way to reuse code. Composition is often better when one object merely uses another.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Define abstraction, encapsulation, inheritance, and polymorphism with short Java examples.
- Likely follow-up: Inheritance is not the only way to reuse code. Composition is often better when one object merely uses another.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Define abstraction, encapsulation, inheritance, and polymorphism with short Java examples.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 07 / 75 | Object Orientation concepts

CONCEPT AND DEFINITIONS

Class, Object, and Their Significance

Simple definition

A class describes what its objects contain and can do. An object is one created instance with its own identity and current state.

Formal / technical definition

A class declaration defines a reference type, its fields, constructors, and methods. Object creation allocates an instance; a reference variable stores a reference value that can designate that object or null.

Notation and how to read it

class = type definition; object = runtime instance; reference = value used to reach instance

How it works

- Identify the class/object and its state/behavior.
- Apply abstraction, encapsulation, inheritance, or polymorphism to the example.
- State one benefit and one design limitation.

Quick map / trace / syntax

```
Lamp class -> new Lamp() -> object on heap
desk reference -> object; field on=true after method call
```

QUICK RECALL

Identify class, reference, object identity, field state, and method behavior in a code trace.

TOPIC 07 / 75 | Object Orientation concepts

CODE EXAMPLE AND EXAM HELP

Class, Object, and Their Significance

Worked code example and output

```
class Lamp { boolean on; void switchOn(){ on=true; } }
Lamp desk = new Lamp(); desk.switchOn();
```

Useful distinction / limitation

A reference variable is not the object itself; assigning one reference to another can make both names reach the same object.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Identify class, reference, object identity, field state, and method behavior in a code trace.
- Likely follow-up: A reference variable is not the object itself; assigning one reference to another can make both names reach the same object.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Identify class, reference, object identity, field state, and method behavior in a code trace.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 08 / 75 | Environment Variables CONCEPT AND DEFINITIONS

Environment Variables and Java Setup

Simple definition

Environment variables and command-line options help the operating system and Java tools locate the JDK, compiler, classes, and libraries.

Formal / technical definition

JAVA_HOME is a commonly used variable pointing to a JDK installation; PATH lets the shell find java and javac. CLASSPATH or the -cp/-classpath option tells launch/compile tools where to search for classes and JARs. Tool-specific environment conventions vary.

Notation and how to read it

JAVA_HOME -> JDK root; PATH -> executable search; -cp -> class/JAR search path

How it works

- Locate the JDK installation and executable path.
- Choose classpath roots for package directories/JARs.
- Run compiler/runtime using the fully qualified main class.

Quick map / trace / syntax

```
shell -> java/javac executable via PATH -> compiler/runtime -> class path lookup
```

QUICK RECALL

Explain JAVA_HOME, PATH, and class path separately; note platform-specific separators.

TOPIC 08 / 75 | Environment Variables

CODE EXAMPLE AND EXAM HELP

Environment Variables and Java Setup

Worked code example and output

```
Windows: set JAVA_HOME=C:\Java\jdk-...
PATH includes %JAVA_HOME%\bin
javac -cp .;lib\db.jar Main.java
Linux/macOS class path separator is usually : instead of ;
```

Useful distinction / limitation

A class path is not the same as PATH: PATH locates commands; class path locates Java classes/resources.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Explain JAVA_HOME, PATH, and class path separately; note platform-specific separators.
- Likely follow-up: A class path is not the same as PATH: PATH locates commands; class path locates Java classes/resources.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Explain JAVA_HOME, PATH, and class path separately; note platform-specific separators.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

Java as a Strongly Typed Language

Simple definition

Java checks that values and operations match declared types, so many incompatible uses are caught by the compiler before the program runs.

Formal / technical definition

Java is statically and strongly typed: variables have declared types, expressions have compile-time types, and conversions follow defined rules. Some failures, such as invalid casts or null dereferences, remain possible at runtime.

Notation and how to read it

declaration: Type name; assignment: name = expression whose type is assignment-compatible

How it works

- Read the declared type and decide whether the name is primitive or reference.
- Apply scope, default/definite assignment, conversion, or promotion rules.
- Check range, index, null, and result-type behavior.

Quick map / trace / syntax

```
declared type -> limits permitted operations and conversions -> compiler checks many errors
```

QUICK RECALL

Give one compile-time type error and one possible runtime type-related failure.

TOPIC 09 / 75 | Data Types, Variables and Array

CODE EXAMPLE AND EXAM HELP

Java as a Strongly Typed Language

Worked code example and output

```
int count = 3;
// count = "three"; // compile-time type error
String label = "three";
```

Useful distinction / limitation

Strong typing does not mean no runtime errors; array bounds, null references, and failed checked casts can still occur.

Common mistakes to avoid

- Strong typing does not prevent all runtime errors.
- A local variable is not automatically initialized like a field.

Quick viva and follow-up cues

- Core answer: Give one compile-time type error and one possible runtime type-related failure.
- Likely follow-up: Strong typing does not mean no runtime errors; array bounds, null references, and failed checked casts can still occur.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Give one compile-time type error and one possible runtime type-related failure.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 10 / 75 | Data Types, Variables and Array CONCEPT AND DEFINITIONS

Primitive Types

Simple definition

Primitive types hold simple values directly, such as whole numbers, decimal numbers, characters, and true/false values.

Formal / technical definition

Java has eight primitive types: byte, short, int, long, float, double, char, and boolean. Integral and floating-point types have defined ranges/representations; boolean is not a numeric type.

Notation and how to read it

Integral: byte/short/int/long/char; floating: float/double; logical: boolean

How it works

- Read the declared type and decide whether the name is primitive or reference.
- Apply scope, default/definite assignment, conversion, or promotion rules.
- Check range, index, null, and result-type behavior.

Quick map / trace / syntax

```
primitive local variable -> typed primitive value
array elements have default values; local variables must be assigned before use
```

QUICK RECALL

List all eight primitives and distinguish numeric types from boolean.

TOPIC 10 / 75 | Data Types, Variables and Array

CODE EXAMPLE AND EXAM HELP

Primitive Types

Worked code example and output

```
byte level = 7;
long population = 9_000_000L;
double rate = 2.5;
char initial = 'A';
boolean active = true;
```

Useful distinction / limitation

char is an unsigned 16-bit UTF-16 code unit, not necessarily a complete Unicode character by itself.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: List all eight primitives and distinguish numeric types from boolean.
- Likely follow-up: char is an unsigned 16-bit UTF-16 code unit, not necessarily a complete Unicode character by itself.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: List all eight primitives and distinguish numeric types from boolean.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 11 / 75 | Data Types, Variables and Array CONCEPT AND DEFINITIONS

Non-Primitive Types

Simple definition

Non-primitive types describe objects. A variable of a reference type holds a reference to an object or null.

Formal / technical definition

Reference types include class, interface, array, and enum types. Their values are references; object fields and array elements have default values when created, while a local reference variable must be definitely assigned before use.

Notation and how to read it

reference variable -> object/array or null; methods are invoked through references when non-null

How it works

- Read the declared type and decide whether the name is primitive or reference.
- Apply scope, default/definite assignment, conversion, or promotion rules.
- Check range, index, null, and result-type behavior.

Quick map / trace / syntax

```
name -> String object
marks -> int array object
student -> null (no object currently designated)
```

QUICK RECALL

Distinguish reference variable, referenced object, and null.

TOPIC 11 / 75 | Data Types, Variables and Array

CODE EXAMPLE AND EXAM HELP

Non-Primitive Types

Worked code example and output

```
String name = "Asha";
int[] marks = {80, 90};
Student student = null;
```

Useful distinction / limitation

Two references can alias the same mutable object; changing it through one reference can be observed through the other.

Common mistakes to avoid

- A reference variable is not the object itself.
- Two references can point to the same object; null points to no object.

Quick viva and follow-up cues

- Core answer: Distinguish reference variable, referenced object, and null.
- Likely follow-up: Two references can alias the same mutable object; changing it through one reference can be observed through the other.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Distinguish reference variable, referenced object, and null.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 12 / 75 | Data Types, Variables and Array CONCEPT AND DEFINITIONS

Wrapper Classes

Simple definition

A wrapper class represents a primitive value as an object, which is useful in collections and APIs that work with objects.

Formal / technical definition

Java provides wrapper types Boolean, Byte, Short, Integer, Long, Character, Float, and Double. Autoboxing converts between a primitive and its wrapper in eligible contexts; unboxing a null wrapper throws NullPointerException.

Notation and how to read it

int <-> Integer; char <-> Character; boolean <-> Boolean; analogous pairs for remaining primitives

How it works

- Read the declared type and decide whether the name is primitive or reference.
- Apply scope, default/definite assignment, conversion, or promotion rules.
- Check range, index, null, and result-type behavior.

Quick map / trace / syntax

```
primitive -> boxing -> wrapper object -> unboxing -> primitive
```

QUICK RECALL

Name the primitive-wrapper pair and describe boxing, unboxing, and null risk.

Wrapper Classes

Worked code example and output

```
Integer boxed = 12; // boxing
int value = boxed; // unboxing
System.out.println(value + 1); // 13
```

Useful distinction / limitation

Wrapper objects are immutable. Do not compare numeric wrappers with == when value equality is intended; use equals or primitive comparison.

Common mistakes to avoid

- Do not use == for wrapper value comparison in general.
- Unboxing a null wrapper causes NullPointerException.

Quick viva and follow-up cues

- Core answer: Name the primitive-wrapper pair and describe boxing, unboxing, and null risk.
- Likely follow-up: Wrapper objects are immutable. Do not compare numeric wrappers with == when value equality is intended; use equals or primitive comparison.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Name the primitive-wrapper pair and describe boxing, unboxing, and null risk.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 13 / 75 | Data Types, Variables and Array CONCEPT AND DEFINITIONS

Variables: Scope and Lifetime

Simple definition

Scope is where a variable name can be used. Lifetime is how long the associated storage/value remains relevant during execution.

Formal / technical definition

Java local variables and parameters are scoped to a block/method invocation; instance fields belong to an object; static fields belong to the class. Local variables have no automatic default initialization, while fields and array elements receive default values.

Notation and how to read it

local/parameter -> method/block; instance field -> object; static field -> class

How it works

- Read the declared type and decide whether the name is primitive or reference.
- Apply scope, default/definite assignment, conversion, or promotion rules.
- Check range, index, null, and result-type behavior.

Quick map / trace / syntax

```
enter method -> local step exists for invocation
object stays reachable -> instance fields available
class loaded -> static members belong to class
```

QUICK RECALL

Classify each name as local, parameter, instance field, or static field and state its scope.

TOPIC 13 / 75 | Data Types, Variables and Array

CODE EXAMPLE AND EXAM HELP

Variables: Scope and Lifetime

Worked code example and output

```
class Counter { static int total; int value; void add(){ int step=1; value += step; } }
// total is class-level; value is per object; step is local to add()
```

Useful distinction / limitation

Do not say all variables are initialized to zero: local variables must be definitely assigned before reading.

Common mistakes to avoid

- Local variables need definite assignment before use.
- Static fields belong to the class; instance fields belong to objects.

Quick viva and follow-up cues

- Core answer: Classify each name as local, parameter, instance field, or static field and state its scope.
- Likely follow-up: Do not say all variables are initialized to zero: local variables must be definitely assigned before reading.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Classify each name as local, parameter, instance field, or static field and state its scope.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 14 / 75 | Data Types, Variables and Array CONCEPT AND DEFINITIONS

Arrays: Declaration and Basic Use

Simple definition

An array stores a fixed number of values of one component type, and each value is reached using an index starting at zero.

Formal / technical definition

An array is a reference type whose object has a fixed length and elements of one component type. Its length is available through the final length field; valid indexes are 0 through length-1.

Notation and how to read it

Type[] name = new Type[length];; access name[index]; last valid index = length - 1

How it works

- Read the declared type and decide whether the name is primitive or reference.
- Apply scope, default/definite assignment, conversion, or promotion rules.
- Check range, index, null, and result-type behavior.

Quick map / trace / syntax

```
index: 0 1 2
value: 78 91 84
```

QUICK RECALL

Trace the index carefully and distinguish length from the last index.

TOPIC 14 / 75 | Data Types, Variables and Array

CODE EXAMPLE AND EXAM HELP

Arrays: Declaration and Basic Use

Worked code example and output

```
int[] scores = {78, 91, 84};
System.out.println(scores[0]);
System.out.println(scores.length);
Output: 78 then 3
```

Useful distinction / limitation

Array length is fixed after creation. Invalid indexes throw ArrayIndexOutOfBoundsException (a runtime exception).

Common mistakes to avoid

- Valid indexes end at length-1, not length.
- Array size is fixed after creation.

Quick viva and follow-up cues

- Core answer: Trace the index carefully and distinguish length from the last index.
- Likely follow-up: Array length is fixed after creation. Invalid indexes throw ArrayIndexOutOfBoundsException (a runtime exception).

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Trace the index carefully and distinguish length from the last index.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 15 / 75 | Data Types, Variables and Array CONCEPT AND DEFINITIONS

Type Conversion and Casting

Simple definition

A conversion changes a value from one type to another. Widening usually fits a broader type; narrowing may lose information and needs an explicit cast.

Formal / technical definition

Java permits specified widening primitive conversions implicitly. Narrowing primitive conversions require casts and may truncate or overflow. Reference casts are checked at runtime; incompatible casts throw ClassCastException.

Notation and how to read it

widening: int -> long/double; narrowing: double -> int requires (int); reference cast: (Subtype) reference

How it works

- Read the declared type and decide whether the name is primitive or reference.
- Apply scope, default/definite assignment, conversion, or promotion rules.
- Check range, index, null, and result-type behavior.

Quick map / trace / syntax

```
widening generally preserves magnitude; narrowing may discard fractional/range information
```

QUICK RECALL

Identify source type, target type, implicit/explicit rule, and any lost information.

TOPIC 15 / 75 | Data Types, Variables and Array

CODE EXAMPLE AND EXAM HELP

Type Conversion and Casting

Worked code example and output

```
int whole = 9;
double precise = whole; // 9.0
double price = 9.8;
int truncated = (int) price; // 9
```

Useful distinction / limitation

Casting a double to int truncates toward zero; it does not round to the nearest integer.

Common mistakes to avoid

- Narrowing can truncate or overflow.
- Casting double to int truncates toward zero; it does not round.

Quick viva and follow-up cues

- Core answer: Identify source type, target type, implicit/explicit rule, and any lost information.
- Likely follow-up: Casting a double to int truncates toward zero; it does not round to the nearest integer.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Identify source type, target type, implicit/explicit rule, and any lost information.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 16 / 75 | Data Types, Variables and Array CONCEPT AND DEFINITIONS

Automatic Type Promotions

Simple definition

In arithmetic, Java may promote smaller numeric values before calculating so the expression uses a wider type.

Formal / technical definition

Unary numeric promotion promotes byte, short, and char operands to int (or preserves wider numeric types). Binary numeric promotion selects double, float, long, or int according to operand types. The resulting expression type may be wider than an assigned variable.

Notation and how to read it

binary numeric promotion: if either double -> double; else float -> float; else long -> long; otherwise int

How it works

- Read the declared type and decide whether the name is primitive or reference.
- Apply scope, default/definite assignment, conversion, or promotion rules.
- Check range, index, null, and result-type behavior.

Quick map / trace / syntax

```
byte + byte -> int
int + double -> double
assignment back to byte -> explicit narrowing cast needed
```

QUICK RECALL

Work out operand promotions, expression type, then assignment compatibility.

TOPIC 16 / 75 | Data Types, Variables and Array

CODE EXAMPLE AND EXAM HELP

Automatic Type Promotions

Worked code example and output

```
byte a = 5, b = 6;
// byte c = a + b; // error: sum is int
byte c = (byte)(a + b);
System.out.println(1 + 2.5); // 3.5
```

Useful distinction / limitation

Boolean is not promoted to a number. Constant-expression assignment rules have special cases; learn the general promotion rule first.

Common mistakes to avoid

- byte/short/char arithmetic commonly promotes to int.
- Check expression type before assigning back to a narrow variable.

Quick viva and follow-up cues

- Core answer: Work out operand promotions, expression type, then assignment compatibility.
- Likely follow-up: Boolean is not promoted to a number. Constant-expression assignment rules have special cases; learn the general promotion rule first.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Work out operand promotions, expression type, then assignment compatibility.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 17 / 75 | Operators

CONCEPT AND DEFINITIONS

Arithmetic Operators

Simple definition

Arithmetic operators calculate with numeric values: add, subtract, multiply, divide, find a remainder, or change a sign.

Formal / technical definition

Java's arithmetic operators include unary +/-, binary +, -, *, /, and %. Integral division truncates toward zero; integer division by zero throws ArithmeticException. Integer overflow wraps according to the fixed-width type.

Notation and how to read it

+ - * / %; unary + and -; integer division discards fractional part

How it works

- Write operand values and compile-time types.
- Apply the operator rule and precedence/parentheses.
- Check result type, boolean outcome, overflow, and side effects where relevant.

Quick map / trace / syntax

```
17 / 5 -> 3 remainder 2
17 % 5 -> 2
```

QUICK RECALL

Identify operand types first; then compute quotient/remainder and check division-by-zero cases.

TOPIC 17 / 75 | Operators

CODE EXAMPLE AND EXAM HELP

Arithmetic Operators

Worked code example and output

```
int a = 17, b = 5;
System.out.println(a / b); // 3
System.out.println(a % b); // 2
System.out.println(-a); // -17
```

Useful distinction / limitation

When either operand is floating point, division can be fractional. Do not apply integer-division intuition to double expressions.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Identify operand types first; then compute quotient/remainder and check division-by-zero cases.
- Likely follow-up: When either operand is floating point, division can be fractional. Do not apply integer-division intuition to double expressions.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Identify operand types first; then compute quotient/remainder and check division-by-zero cases.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 18 / 75 | Operators

CONCEPT AND DEFINITIONS

Bitwise Operators

Simple definition

Bitwise operators work on the individual bits of integral values; shifts move bit patterns left or right.

Formal / technical definition

For integral operands, &, |, ^, and ~ perform bitwise operations; <<, >>, and >>> shift bits. For boolean operands, &, |, and ^ can combine boolean values without short-circuiting, while && and || short-circuit.

Notation and how to read it

& AND; | OR; ^ XOR; ~ complement; << left shift; >> sign-extending right; >>> zero-fill right

How it works

- Write operand values and compile-time types.
- Apply the operator rule and precedence/parentheses.
- Check result type, boolean outcome, overflow, and side effects where relevant.

Quick map / trace / syntax

```
0110 & 0011 = 0010
0110 ^ 0011 = 0101
```

QUICK RECALL

Convert operands to binary and apply the operator column by column.

TOPIC 18 / 75 | Operators

CODE EXAMPLE AND EXAM HELP

Bitwise Operators

Worked code example and output

```
int a = 6; // 0110
int b = 3; // 0011
System.out.println(a & b); // 2 (0010)
System.out.println(a ^ b); // 5 (0101)
System.out.println(a << 1); // 12
```

Useful distinction / limitation

Use &&/|| for short-circuit boolean conditions; bitwise boolean operators evaluate both operands.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Convert operands to binary and apply the operator column by column.
- Likely follow-up: Use &&/|| for short-circuit boolean conditions; bitwise boolean operators evaluate both operands.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Convert operands to binary and apply the operator column by column.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 19 / 75 | Operators

CONCEPT AND DEFINITIONS

Relational Operators

Simple definition

Relational operators compare two values and produce a boolean result: true or false.

Formal / technical definition

Equality operators are == and !=; ordered comparisons are <, <=, >, and >= for compatible primitive numeric/character operands. Reference equality compares identity, not object content; equals is commonly used for content equality.

Notation and how to read it

comparison expression -> boolean; reference == checks same object identity

How it works

- Write operand values and compile-time types.
- Apply the operator rule and precedence/parentheses.
- Check result type, boolean outcome, overflow, and side effects where relevant.

Quick map / trace / syntax

```
5 >= 5 -> true
separate String objects: a == b false; a.equals(b) true
```

QUICK RECALL

State the operand types and whether the comparison is value equality or reference identity.

TOPIC 19 / 75 | Operators

CODE EXAMPLE AND EXAM HELP

Relational Operators

Worked code example and output

```
int x = 5;
System.out.println(x >= 5); // true
String a = new String("Java");
String b = new String("Java");
System.out.println(a == b); // false
System.out.println(a.equals(b)); // true
```

Useful distinction / limitation

For String value comparison, use equals (or a suitable comparison method), not ==.

Common mistakes to avoid

- Reference == compares identity, not String content.
- Use equals when object value equality is intended.

Quick viva and follow-up cues

- Core answer: State the operand types and whether the comparison is value equality or reference identity.
- Likely follow-up: For String value comparison, use equals (or a suitable comparison method), not ==.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: State the operand types and whether the comparison is value equality or reference identity.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 20 / 75 | Operators

CONCEPT AND DEFINITIONS

Assignment Operators

Simple definition

Assignment stores a value in a variable. Compound assignments combine an operation with storing the result back.

Formal / technical definition

The simple assignment operator is =. Compound forms include +=, -=, *=, /=, %=, bitwise and shift assignments. Assignment is right-associative; compound assignment includes an implicit cast to the left-hand variable type after the operation.

Notation and how to read it

x = value; x += y; x <<= k; compound x op= y behaves like x = (type of x)(x op y)

How it works

- Write operand values and compile-time types.
- Apply the operator rule and precedence/parentheses.
- Check result type, boolean outcome, overflow, and side effects where relevant.

Quick map / trace / syntax

```
right-hand expression -> assignment-compatible value -> variable updated
```

QUICK RECALL

Trace the old value, expression result, any compound narrowing, and the stored value.

TOPIC 20 / 75 | Operators

CODE EXAMPLE AND EXAM HELP

Assignment Operators

Worked code example and output

```
int n = 4;
n += 3; // n = 7
byte level = 10;
level += 1; // legal compound assignment
```

Useful distinction / limitation

The implicit cast in compound assignment can narrow and lose information; it is not identical in type checking to writing x = x + y.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Trace the old value, expression result, any compound narrowing, and the stored value.
- Likely follow-up: The implicit cast in compound assignment can narrow and lose information; it is not identical in type checking to writing x = x + y.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Trace the old value, expression result, any compound narrowing, and the stored value.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 21 / 75 | Operators

CONCEPT AND DEFINITIONS

The ? Operator (Conditional Operator)

Simple definition

The conditional operator chooses one of two expressions based on a boolean condition; it is a compact expression form of a two-way choice.

Formal / technical definition

The conditional expression has syntax condition ? expressionIfTrue : expressionIfFalse. The condition must be boolean, and the two result expressions must have types that can produce a compatible conditional-expression type.

Notation and how to read it

condition ? valueWhenTrue : valueWhenFalse

How it works

- Write operand values and compile-time types.
- Apply the operator rule and precedence/parentheses.
- Check result type, boolean outcome, overflow, and side effects where relevant.

Quick map / trace / syntax

```
condition true -> left expression
condition false -> right expression
only the selected result expression is evaluated
```

QUICK RECALL

Evaluate the condition first, then evaluate only the selected branch and check the result type.

TOPIC 21 / 75 | Operators

CODE EXAMPLE AND EXAM HELP

The ? Operator (Conditional Operator)

Worked code example and output

```
int a = 8, b = 5;
int larger = (a > b) ? a : b;
System.out.println(larger); // 8
```

Useful distinction / limitation

Nested conditional expressions can be hard to read; use if/else when several branches or side effects are involved.

Common mistakes to avoid

- Condition must be boolean and both result expressions need a compatible type.
- Do not nest many conditionals when if/else is clearer.

Quick viva and follow-up cues

- Core answer: Evaluate the condition first, then evaluate only the selected branch and check the result type.
- Likely follow-up: Nested conditional expressions can be hard to read; use if/else when several branches or side effects are involved.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Evaluate the condition first, then evaluate only the selected branch and check the result type.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 22 / 75 | Operators

CONCEPT AND DEFINITIONS

Operator Precedence

Simple definition

Precedence decides which operator binds first. Parentheses make the intended grouping explicit.

Formal / technical definition

Java expressions follow defined precedence and associativity rules. Common order from high to low: postfix/unary, multiplicative, additive, shifts, relational, equality, bitwise, logical, conditional, assignment. Most binary operators associate left-to-right; assignment and conditional expressions associate right-to-left.

Notation and how to read it

multiplication before addition; relational before &&; && before ||; conditional before assignment

How it works

- Write operand values and compile-time types.
- Apply the operator rule and precedence/parentheses.
- Check result type, boolean outcome, overflow, and side effects where relevant.

Quick map / trace / syntax

```
2 + 3 * 4 = 2 + (3 * 4) = 14
parentheses override default grouping
```

QUICK RECALL

Add parentheses to show the parse tree, then evaluate from the innermost expression outward.

TOPIC 22 / 75 | Operators

CODE EXAMPLE AND EXAM HELP

Operator Precedence

Worked code example and output

```
int result = 2 + 3 * 4;
System.out.println(result); // 14
System.out.println((2 + 3) * 4); // 20
```

Useful distinction / limitation

Precedence controls parsing, not necessarily evaluation order for every operand side effect; avoid clever expressions.

Common mistakes to avoid

- Use parentheses to make grouping obvious.
- Precedence and evaluation order are different concepts.

Quick viva and follow-up cues

- Core answer: Add parentheses to show the parse tree, then evaluate from the innermost expression outward.
- Likely follow-up: Precedence controls parsing, not necessarily evaluation order for every operand side effect; avoid clever expressions.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Add parentheses to show the parse tree, then evaluate from the innermost expression outward.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 23 / 75 | Control Statements CONCEPT AND DEFINITIONS

Selection Statements

Simple definition

Selection statements choose which block runs based on a condition or a value.

Formal / technical definition

Java selection constructs include if, if-else, and switch. An if condition must be boolean. A switch selects a matching case according to permitted selector types and syntax for the target Java version.

Notation and how to read it

if (boolean) { ... } else { ... }; switch (selector) { case ...: ...; }

How it works

- Evaluate conditions in order.
- Run the first matching branch or inspect switch fall-through.
- Check boundary/default behavior.

Quick map / trace / syntax

```
test first condition -> first true branch runs -> later branches skipped
```

QUICK RECALL

Trace conditions in order and state exactly one selected path or any switch fall-through.

TOPIC 23 / 75 | Control Statements

CODE EXAMPLE AND EXAM HELP

Selection Statements

Worked code example and output

```
int score = 74;
char grade;
if (score >= 90) grade = 'A';
else if (score >= 60) grade = 'B';
else grade = 'C';
// grade is B
```

Useful distinction / limitation

In classic switch, missing break can cause fall-through. Test boundaries and include a default case when useful.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Trace conditions in order and state exactly one selected path or any switch fall-through.
- Likely follow-up: In classic switch, missing break can cause fall-through. Test boundaries and include a default case when useful.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Trace conditions in order and state exactly one selected path or any switch fall-through.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 24 / 75 | Control Statements CONCEPT AND DEFINITIONS

Iteration Statements

Simple definition

Iteration statements repeat a block while a condition holds or for each item in a sequence.

Formal / technical definition

Java provides while, do-while, basic for, and enhanced for statements. while checks before each iteration; do-while checks after the body; enhanced for iterates over arrays or Iterable values.

Notation and how to read it

while(condition); do { } while(condition); for(init; test; update); for(Type x : array)

How it works

- Record initialization and loop test.
- Trace body and update one iteration at a time.
- Stop at the first failed condition or jump statement.

Quick map / trace / syntax

```
initialize -> test -> body -> update -> repeat -> exit when test false
```

QUICK RECALL

Record initialization, condition, body effect, update, and final failed condition in a trace table.

TOPIC 24 / 75 | Control Statements

CODE EXAMPLE AND EXAM HELP

Iteration Statements

Worked code example and output

```
int sum = 0;
for (int n = 1; n <= 3; n++) sum += n;
System.out.println(sum); // 6
```

Useful distinction / limitation

A loop can be infinite if its state never reaches the exit condition. Enhanced for is not the usual way to replace array elements by index.

Common mistakes to avoid

- Check loop initialization, condition, and update for nontermination.
- An enhanced for loop provides values, not a convenient write-back index.

Quick viva and follow-up cues

- Core answer: Record initialization, condition, body effect, update, and final failed condition in a trace table.
- Likely follow-up: A loop can be infinite if its state never reaches the exit condition. Enhanced for is not the usual way to replace array elements by index.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Record initialization, condition, body effect, update, and final failed condition in a trace table.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 25 / 75 | Control Statements CONCEPT AND DEFINITIONS

Jump Statements

Simple definition

Jump statements change the normal path through a method or loop: exit a loop, skip an iteration, or return from a method.

Formal / technical definition

Java jump statements include break, continue, and return. Unlabeled break exits the nearest loop or switch; continue advances the nearest loop; return terminates the current method and may provide a value.

Notation and how to read it

break -> leave loop/switch; continue -> next loop iteration; return [value] -> leave method

How it works

- Identify the nearest loop or method target.
- Mark which statements are skipped or executed next.
- Trace the final value/return path.

Quick map / trace / syntax

```
continue skips remaining body statements for this turn
break skips remaining loop turns
```

QUICK RECALL

Identify the target loop/method and list which statements are skipped or still run.

TOPIC 25 / 75 | Control Statements

CODE EXAMPLE AND EXAM HELP

Jump Statements

Worked code example and output

```
for (int n=1; n<=5; n++) {
if (n==2) continue;
if (n==5) break;
System.out.print(n + " ");
}
// prints 1 3 4
```

Useful distinction / limitation

An unlabeled jump affects the nearest enclosing target. Labeled jumps exist but should be used sparingly.

Common mistakes to avoid

- Unlabeled break/continue affects the nearest target.
- return exits the method, not just a loop.

Quick viva and follow-up cues

- Core answer: Identify the target loop/method and list which statements are skipped or still run.
- Likely follow-up: An unlabeled jump affects the nearest enclosing target. Labeled jumps exist but should be used sparingly.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Identify the target loop/method and list which statements are skipped or still run.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 26 / 75 | Introducing Classes

CONCEPT AND DEFINITIONS

Class Fundamentals

Simple definition

A class is a blueprint that declares fields for state, methods for behavior, and constructors for initializing new objects.

Formal / technical definition

A class declaration defines a reference type with members, access modifiers, constructors, and optional inheritance/implementation clauses. A constructor has the class name and no return type; it initializes an instance during creation.

Notation and how to read it

class Name { fields; constructors; methods; nested types }

How it works

- Name the class, members, constructor, or type relationship.
- Trace creation/member lookup/call using the declared and runtime types.
- State the resulting object state or method behavior.

Quick map / trace / syntax

```
declaration -> class type
new Point(3,4) -> constructor initializes x=3,y=4
```

QUICK RECALL

Label fields, constructor, parameters, this, return type, and method in a small class.

TOPIC 26 / 75 | Introducing Classes

CODE EXAMPLE AND EXAM HELP

Class Fundamentals

Worked code example and output

```
class Point {
int x, y;
Point(int x, int y) { this.x=x; this.y=y; }
int distanceSquared() { return x*x + y*y; }
}
```

Useful distinction / limitation

A class with no declared constructor receives a default no-argument constructor; declaring any constructor removes that implicit default.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Label fields, constructor, parameters, this, return type, and method in a small class.
- Likely follow-up: A class with no declared constructor receives a default no-argument constructor; declaring any constructor removes that implicit default.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Label fields, constructor, parameters, this, return type, and method in a small class.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 27 / 75 | Introducing Classes

CONCEPT AND DEFINITIONS

Creating and Operating Objects

Simple definition

The new operator creates an object and calls a constructor; a reference then lets the program use its fields and methods.

Formal / technical definition

Class-instance creation evaluates constructor arguments, allocates and initializes an instance, and returns a reference. Instance member access uses dot notation; static members belong to the class rather than one instance.

Notation and how to read it

Type ref = new Type(arguments); ref.field; ref.method(arguments); ClassName.staticMember

How it works

- Name the class, members, constructor, or type relationship.
- Trace creation/member lookup/call using the declared and runtime types.
- State the resulting object state or method behavior.

Quick map / trace / syntax

```
new -> object + constructor -> reference -> method updates instance state
```

QUICK RECALL

Trace construction, reference assignment, method call, and final object field values.

TOPIC 27 / 75 | Introducing Classes

CODE EXAMPLE AND EXAM HELP

Creating and Operating Objects

Worked code example and output

```
class Lamp { boolean on; void turnOn(){ on=true; } }
Lamp desk = new Lamp();
desk.turnOn();
System.out.println(desk.on); // true
```

Useful distinction / limitation

Dereferencing null throws NullPointerException. Use constructors/methods to keep object state valid.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Trace construction, reference assignment, method call, and final object field values.
- Likely follow-up: Dereferencing null throws NullPointerException. Use constructors/methods to keep object state valid.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Trace construction, reference assignment, method call, and final object field values.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 28 / 75 | Introducing Classes

CONCEPT AND DEFINITIONS

Methods and Defining Methods

Simple definition

A method is a named block of code. Its signature tells callers how to invoke it, and its body performs work or returns a result.

Formal / technical definition

A method declaration includes modifiers, result type, name, formal parameters, optional throws clause, and body. A void method returns no value; a non-void method must return a compatible value on every normally completing path.

Notation and how to read it

[modifiers] ResultType name(ParameterType name, ...) { body }

How it works

- Name the class, members, constructor, or type relationship.
- Trace creation/member lookup/call using the declared and runtime types.
- State the resulting object state or method behavior.

Quick map / trace / syntax

```
call -> arguments bind to parameters -> statements run -> result returned (or void completion)
```

QUICK RECALL

Identify return type, method name, parameters, call site, and returned value.

TOPIC 28 / 75 | Introducing Classes

CODE EXAMPLE AND EXAM HELP

Methods and Defining Methods

Worked code example and output

```
static int square(int value) {
return value * value;
}
System.out.println(square(5)); // 25
```

Useful distinction / limitation

A method's return type is not part of its overload signature. Every path in a non-void method must return.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Identify return type, method name, parameters, call site, and returned value.
- Likely follow-up: A method's return type is not part of its overload signature. Every path in a non-void method must return.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Identify return type, method name, parameters, call site, and returned value.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 29 / 75 | Introducing Classes

CONCEPT AND DEFINITIONS

Argument Passing Mechanism

Simple definition

Java always passes arguments by value. For an object parameter, the copied value is a reference, so the method can mutate that object but cannot replace the caller's reference variable.

Formal / technical definition

At invocation, each actual argument value initializes a corresponding formal parameter. Primitive values are copied; reference values are copied as well. Both reference variables may designate the same mutable object.

Notation and how to read it

primitive: copied primitive; reference: copied reference value, not a deep copy of object

How it works

- Copy each actual argument value to its parameter.
- For references, draw caller and parameter arrows to the object.
- Separate object mutation from parameter reassignment.

Quick map / trace / syntax

```
caller a -> array <- parameter a
parameter changes element -> visible to caller
parameter reassignment -> caller reference unchanged
```

QUICK RECALL

Trace primitive reassignment, object mutation, and parameter reassignment as separate effects.

TOPIC 29 / 75 | Introducing Classes

CODE EXAMPLE AND EXAM HELP

Argument Passing Mechanism

Worked code example and output

```
static void change(int n, int[] a) { n=9; a[0]=9; a=new int[]{0}; }
int x=1; int[] data={1}; change(x,data);
// x=1, data[0]=9
```

Useful distinction / limitation

Java is not pass-by-reference. Explain aliasing to describe visible object mutation without changing the rule.

Common mistakes to avoid

- Java always passes by value, including reference values.
- Object mutation may be visible, but reassigning the parameter does not replace caller's reference.

Quick viva and follow-up cues

- Core answer: Trace primitive reassignment, object mutation, and parameter reassignment as separate effects.
- Likely follow-up: Java is not pass-by-reference. Explain aliasing to describe visible object mutation without changing the rule.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Trace primitive reassignment, object mutation, and parameter reassignment as separate effects.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 30 / 75 | Introducing Classes

CONCEPT AND DEFINITIONS

Method Overloading

Simple definition

Overloading lets one class use the same method name for different parameter lists, so related operations have a familiar name.

Formal / technical definition

Methods are overloaded when names match but parameter signatures differ by number, types, or order. Return type alone cannot distinguish overloads. The compiler selects an applicable overload using compile-time argument types and conversion rules.

Notation and how to read it

same name + distinct parameter list; return type alone is insufficient

How it works

- Name the class, members, constructor, or type relationship.
- Trace creation/member lookup/call using the declared and runtime types.
- State the resulting object state or method behavior.

Quick map / trace / syntax

```
area(4) -> area(int)
area(2.0,3.0) -> area(double,double)
```

QUICK RECALL

Compare parameter lists and trace overload selection from the argument types.

TOPIC 30 / 75 | Introducing Classes

CODE EXAMPLE AND EXAM HELP

Method Overloading

Worked code example and output

```
static int area(int side){ return side*side; }
static double area(double w, double h){ return w*h; }
System.out.println(area(4)); // 16
```

Useful distinction / limitation

Ambiguous calls can fail compilation when overload resolution cannot select a most-specific method.

Common mistakes to avoid

- Changing only the return type does not create a new overload.
- Overload selection uses compile-time argument types.

Quick viva and follow-up cues

- Core answer: Compare parameter lists and trace overload selection from the argument types.
- Likely follow-up: Ambiguous calls can fail compilation when overload resolution cannot select a most-specific method.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Compare parameter lists and trace overload selection from the argument types.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 31 / 75 | Introducing Classes CONCEPT AND DEFINITIONS

Recursion

Simple definition

Recursion happens when a method solves a problem by calling itself on a smaller case, until a simple base case is reached.

Formal / technical definition

A recursive method invocation creates a new activation frame with its own parameters and local variables. A correct recursive algorithm has a base case and a progress step that moves toward it.

Notation and how to read it

problem(n) -> n * problem(n-1); base: problem(1)=1

How it works

- Write the base case.
- Expand calls until the base case; show each frame's parameters.
- Unwind return values in reverse order.

Quick map / trace / syntax

```
factorial(4) -> 4*factorial(3) -> 4*3*2*1 -> 24
```

QUICK RECALL

Show the base case, recursive call, decreasing measure, and return-value unwind.

TOPIC 31 / 75 | Introducing Classes

CODE EXAMPLE AND EXAM HELP

Recursion

Worked code example and output

```
static int factorial(int n) {
if (n <= 1) return 1;
return n * factorial(n-1);
}
System.out.println(factorial(4)); // 24
```

Useful distinction / limitation

Missing or unreachable base cases can cause unbounded recursion and StackOverflowError. Validate inputs such as negative n.

Common mistakes to avoid

- Every recursive path needs a reachable base case and progress.
- Large recursion can exhaust the thread stack.

Quick viva and follow-up cues

- Core answer: Show the base case, recursive call, decreasing measure, and return-value unwind.
- Likely follow-up: Missing or unreachable base cases can cause unbounded recursion and StackOverflowError. Validate inputs such as negative n.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Show the base case, recursive call, decreasing measure, and return-value unwind.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

Nested and Inner Classes

Simple definition

A class may be declared inside another class. A static nested class does not need an outer object; a non-static inner class is associated with an outer instance.

Formal / technical definition

Member nested classes are declared in another class. A static nested class has no enclosing-instance reference; a non-static inner class has an enclosing instance and can access its members subject to access rules. Local classes are declared in a block.

Notation and how to read it

Outer.Inner: non-static inner; Outer.Nested: static nested; local class: method/block scope

How it works

- Name the class, members, constructor, or type relationship.
- Trace creation/member lookup/call using the declared and runtime types.
- State the resulting object state or method behavior.

Quick map / trace / syntax

```
Outer instance -> Inner instance can access its enclosing object's members
static nested type is created without Outer instance
```

QUICK RECALL

State whether the nested type is static and show how its instance is created.

TOPIC 32 / 75 | Introducing Classes

CODE EXAMPLE AND EXAM HELP

Nested and Inner Classes

Worked code example and output

```
class Outer {
private int value=7;
class Inner { int get(){ return value; } }
static class Nested { }
}
Outer.Inner i = new Outer().new Inner();
```

Useful distinction / limitation

Inner creation syntax requires an enclosing instance; static nested classes do not have an implicit outer-instance link.

Common mistakes to avoid

- Static nested classes do not require an outer object.
- A non-static inner class instance is tied to an enclosing instance.

Quick viva and follow-up cues

- Core answer: State whether the nested type is static and show how its instance is created.
- Likely follow-up: Inner creation syntax requires an enclosing instance; static nested classes do not have an implicit outer-instance link.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: State whether the nested type is static and show how its instance is created.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 33 / 75 | Introducing Classes CONCEPT AND DEFINITIONS

Anonymous Classes

Simple definition

An anonymous class creates a one-off implementation or subclass right where it is needed, without writing a separate class name.

Formal / technical definition

An anonymous class is an unnamed class-instance creation expression that declares a class body, typically implementing an interface or extending a class. It can capture local variables that are final or effectively final.

Notation and how to read it

new InterfaceOrBase(args) { override methods; };

How it works

- Name the class, members, constructor, or type relationship.
- Trace creation/member lookup/call using the declared and runtime types.
- State the resulting object state or method behavior.

Quick map / trace / syntax

```
one-time behavior -> anonymous class body -> object assigned to interface/base reference
```

QUICK RECALL

Identify the base interface/class, overridden behavior, and the reference type of the result.

TOPIC 33 / 75 | Introducing Classes

CODE EXAMPLE AND EXAM HELP

Anonymous Classes

Worked code example and output

```
Runnable task = new Runnable() {
public void run() { System.out.println("working"); }
};
new Thread(task).start();
```

Useful distinction / limitation

For modern Java, a lambda is shorter for a functional interface; the anonymous-class form remains useful for multiple methods/state or legacy syllabus code.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Identify the base interface/class, overridden behavior, and the reference type of the result.
- Likely follow-up: For modern Java, a lambda is shorter for a functional interface; the anonymous-class form remains useful for multiple methods/state or legacy syllabus code.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Identify the base interface/class, overridden behavior, and the reference type of the result.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 34 / 75 | Introducing Classes

CONCEPT AND DEFINITIONS

Abstract Classes and Interfaces

Simple definition

An abstract class is a partly implemented parent. An interface describes behavior a class agrees to provide; one class can implement several interfaces.

Formal / technical definition

An abstract class cannot be instantiated and may contain abstract and concrete methods, fields, and constructors. An interface defines a reference type; classes implement its abstract methods and may inherit default/static methods. A class extends one class but can implement multiple interfaces.

Notation and how to read it

class Child extends OneClass implements InterfaceA, InterfaceB

How it works

- Name the class, members, constructor, or type relationship.
- Trace creation/member lookup/call using the declared and runtime types.
- State the resulting object state or method behavior.

Quick map / trace / syntax

```
abstract base -> shared state/implementation
interface -> behavioral contract; implementing class supplies required methods
```

QUICK RECALL

Compare instantiation, constructors/state, inheritance count, and typical purpose.

TOPIC 34 / 75 | Introducing Classes

CODE EXAMPLE AND EXAM HELP

Abstract Classes and Interfaces

Worked code example and output

```
abstract class Shape { abstract double area(); }
interface Printable { void print(); }
class Circle extends Shape implements Printable {
double area(){ return 3.14; }
public void print(){ System.out.println(area()); }
}
```

Useful distinction / limitation

An interface is not simply an abstract class with no methods; interfaces can define constants and several kinds of methods under Java rules.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Compare instantiation, constructors/state, inheritance count, and typical purpose.
- Likely follow-up: An interface is not simply an abstract class with no methods; interfaces can define constants and several kinds of methods under Java rules.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Compare instantiation, constructors/state, inheritance count, and typical purpose.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 35 / 75 | Introducing Classes

CONCEPT AND DEFINITIONS

Garbage Collection and Object Reachability

Simple definition

Java reclaims heap memory for objects that can no longer be reached by a running program; the programmer does not manually free each object.

Formal / technical definition

The JVM performs automatic memory management. An object is eligible for collection when it is no longer strongly reachable from live program roots, but collection time is nondeterministic and implementation-dependent. System.gc() is only a request/hint.

Notation and how to read it

reachable object -> retained; unreachable object -> eligible, not guaranteed to be collected immediately

How it works

- Track live references from local/static roots and reachable objects.
- Decide when an object becomes eligible, not when collection must occur.
- Use explicit cleanup for external resources.

Quick map / trace / syntax

```
stack/static/live object references -> reachable object graph -> heap
remove last path -> object eligible for GC
```

QUICK RECALL

Explain reachability and why eligibility is not a promise of immediate collection.

TOPIC 35 / 75 | Introducing Classes

CODE EXAMPLE AND EXAM HELP

Garbage Collection and Object Reachability

Worked code example and output

```
Node first = new Node();
first = null; // if no other reference reaches the Node, it may become eligible for collection
```

Useful distinction / limitation

Garbage collection reclaims memory, not file handles or other resources at a predictable time; use explicit close/try-with-resources for those.

Common mistakes to avoid

- Unreachable means eligible, not collected immediately.
- Garbage collection is not deterministic resource cleanup.

Quick viva and follow-up cues

- Core answer: Explain reachability and why eligibility is not a promise of immediate collection.
- Likely follow-up: Garbage collection reclaims memory, not file handles or other resources at a predictable time; use explicit close/try-with-resources for those.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Explain reachability and why eligibility is not a promise of immediate collection.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 36 / 75 | Introducing Classes

CONCEPT AND DEFINITIONS

finalize() Method

Simple definition

finalize() was an old cleanup hook that the garbage collector could call before reclaiming an object, but its timing was unreliable.

Formal / technical definition

Object.finalize() was a protected method invoked, if at all, by a finalizer mechanism before reclamation. Invocation order/timing is not guaranteed; finalization is deprecated for removal in current Java and should not be used for resource management.

Notation and how to read it

finalize() != constructor/destructor; current replacement: explicit close()/AutoCloseable or a supported Cleaner use case

How it works

- Track live references from local/static roots and reachable objects.
- Decide when an object becomes eligible, not when collection must occur.
- Use explicit cleanup for external resources.

Quick map / trace / syntax

```
object becomes unreachable -> finalizer may run later or not before shutdown -> memory eventually reclaimed
```

QUICK RECALL

Explain why finalize is unreliable and give deterministic resource cleanup as the alternative.

TOPIC 36 / 75 | Introducing Classes

CODE EXAMPLE AND EXAM HELP

finalize() Method

Worked code example and output

```
Legacy form: protected void finalize() throws Throwable { /* old cleanup example */ }
Modern resource pattern: try (FileInputStream in = new FileInputStream(path)) { ... }
```

Useful distinction / limitation

As of JDK 27, finalization remains deprecated for removal; do not rely on finalize() for correctness or cleanup.

Common mistakes to avoid

- Finalization order/time is not guaranteed.
- Finalize is deprecated for removal; use explicit/managed cleanup.

Quick viva and follow-up cues

- Core answer: Explain why finalize is unreliable and give deterministic resource cleanup as the alternative.
- Likely follow-up: As of JDK 27, finalization remains deprecated for removal; do not rely on finalize() for correctness or cleanup.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Explain why finalize is unreliable and give deterministic resource cleanup as the alternative.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

MODULE-I COMPLETE

Pause. Review. Then continue.

MODULE-I established Java's execution model, type rules, operators, control flow, and class fundamentals.

Before you begin the next module

- Explain one important rule without looking at the notes.
- Redo a short code trace and identify every output line.
- Use the next roadmap to preview the topics and their connections.

NEXT MODULE STARTS NEXT

MODULE-II - Inheritance, Packages, Exception Handling, Threads, Arrays and Strings

QUICK RECALL

Short regular practice is more useful than memorizing syntax without tracing it.

MODULE-II - what to study and in what order

Module goal

Reuse behavior through inheritance/packages, handle failures clearly, and coordinate concurrent work safely.

Syllabus map - exact topic order

- Inheritance: Use and Benefits of Inheritance in OOP; Types of Inheritance in Java; Inheriting Data Members and Methods; Role of Constructors in Inheritance; Overloading and Polymorphism in Inheritance.
- Package: Organizing Classes and Interfaces in Packages; Package as Access Protection; Defining Packages; CLASSPATH Setting for Packages.
- Exception Handling: The Idea Behind Exceptions; Exceptions and Errors: Types; Control Flow in Exceptions and JVM Reaction; try, catch, and finally; throw and throws; Inbuilt and User-Defined Exceptions; Checked and Unchecked Exceptions.
- Thread: Understanding Threads and Need for Multithreaded Programming; Thread Life-Cycle; Thread Priorities; Synchronizing Threads; Inter-Communication of Threads.
- Array &String: Defining, Initializing, and Accessing an Array; Multidimensional Arrays; Operations on String.

Recommended study order

- Learn inheritance and member/constructor rules before packages.
- Trace exceptions through handlers and method callers.
- Study thread states and synchronization; finish with arrays and String operations.

QUICK RECALL

Explain one race condition and how a synchronized block protects shared state.

TOPIC 37 / 75 | Inheritance

CONCEPT AND DEFINITIONS

Use and Benefits of Inheritance in OOP

Simple definition

Inheritance lets a child class reuse and specialize behavior from a parent class, which can reduce duplication when the relationship is truly an is-a relationship.

Formal / technical definition

A subclass derives from a superclass and inherits accessible members. It may add members or override eligible instance methods. Inheritance supports subtype polymorphism and shared behavior but increases coupling.

Notation and how to read it

class Subclass extends Superclass { ... }

How it works

- Draw superclass/subclass relationships and identify the declared reference type.
- Check member access and constructor chain.
- Separate compile-time overload resolution from runtime override dispatch.

Quick map / trace / syntax

```
Animal behavior -> shared in Dog
Dog adds bark() and may specialize inherited methods
```

QUICK RECALL

Name inherited behavior, subclass specialization, and one case where composition is a better fit.

TOPIC 37 / 75 | Inheritance

CODE EXAMPLE AND EXAM HELP

Use and Benefits of Inheritance in OOP

Worked code example and output

```
class Animal { void eat(){ System.out.println("eating"); } }
class Dog extends Animal { void bark(){ } }
Dog d = new Dog(); d.eat();
```

Useful distinction / limitation

Use composition when the relationship is has-a rather than is-a; reuse alone is not enough reason to inherit.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Name inherited behavior, subclass specialization, and one case where composition is a better fit.
- Likely follow-up: Use composition when the relationship is has-a rather than is-a; reuse alone is not enough reason to inherit.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Name inherited behavior, subclass specialization, and one case where composition is a better fit.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 38 / 75 | Inheritance CONCEPT AND DEFINITIONS

Types of Inheritance in Java

Simple definition

Java classes can form single, multilevel, and hierarchical inheritance. A class cannot extend two classes, but it can implement multiple interfaces.

Formal / technical definition

Single inheritance gives a class one direct superclass; multilevel inheritance forms a chain; hierarchical inheritance has multiple subclasses of a parent. Java does not support multiple inheritance of classes, avoiding ambiguous inherited class state/implementation; multiple interface inheritance is supported.

Notation and how to read it

class C extends one-class; class C implements InterfaceA, InterfaceB

How it works

- Draw superclass/subclass relationships and identify the declared reference type.
- Check member access and constructor chain.
- Separate compile-time overload resolution from runtime override dispatch.

Quick map / trace / syntax

```
A -> B -> C (multilevel)
A -> B and A -> D (hierarchical)
```

QUICK RECALL

Draw the inheritance shape and distinguish class inheritance from interface implementation.

TOPIC 38 / 75 | Inheritance
CODE EXAMPLE AND EXAM HELP

Types of Inheritance in Java

Worked code example and output

```
class A {}
class B extends A {} // single
class C extends B {} // multilevel
class D extends A {} // hierarchical
class E implements X, Y {} // multiple interfaces
```

Useful distinction / limitation

Do not say Java has no multiple inheritance at all; it forbids multiple superclass inheritance but permits implementing multiple interfaces.

Common mistakes to avoid

- Java has no multiple superclass inheritance.
- A class may implement multiple interfaces.

Quick viva and follow-up cues

- Core answer: Draw the inheritance shape and distinguish class inheritance from interface implementation.
- Likely follow-up: Do not say Java has no multiple inheritance at all; it forbids multiple superclass inheritance but permits implementing multiple interfaces.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Draw the inheritance shape and distinguish class inheritance from interface implementation.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 39 / 75 | Inheritance

CONCEPT AND DEFINITIONS

Inheriting Data Members and Methods

Simple definition

A subclass can use accessible inherited fields and methods, while private superclass members remain hidden from direct subclass access.

Formal / technical definition

Member access depends on access modifier, package, and subclass relationship. Public and protected members are accessible under their rules; package-private members are available in the same package; private members are accessible only within their declaring top-level class (including nested access rules). Inherited members are looked up through the class hierarchy.

Notation and how to read it

public; protected; package-private (no modifier); private

How it works

- Draw superclass/subclass relationships and identify the declared reference type.
- Check member access and constructor chain.
- Separate compile-time overload resolution from runtime override dispatch.

Quick map / trace / syntax

```
Savings object -> inherited protected balance
private pin remains encapsulated in Account
```

QUICK RECALL

Apply access rules using both the member modifier and the subclass's package/location.

TOPIC 39 / 75 | Inheritance

CODE EXAMPLE AND EXAM HELP

Inheriting Data Members and Methods

Worked code example and output

```
class Account { protected int balance=10; private int pin=1234; }
class Savings extends Account { void show(){ System.out.println(balance); /* pin is not directly accessible */ } }
```

Useful distinction / limitation

A private field still exists as part of superclass state; subclass methods should use a suitable protected/public method if access is intended.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Apply access rules using both the member modifier and the subclass's package/location.
- Likely follow-up: A private field still exists as part of superclass state; subclass methods should use a suitable protected/public method if access is intended.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Apply access rules using both the member modifier and the subclass's package/location.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 40 / 75 | Inheritance

CONCEPT AND DEFINITIONS

Role of Constructors in Inheritance

Simple definition

A constructor initializes the superclass part first, then the subclass part. A child constructor can choose which parent constructor runs.

Formal / technical definition

Constructors are not inherited or overridden. If a constructor body omits an explicit constructor invocation, Java inserts a no-argument super() call when applicable. A superclass constructor must complete before the subclass initialization body proceeds.

Notation and how to read it

constructor chain: Object/superclass -> subclass; explicit super(args) is first statement

How it works

- Draw superclass/subclass relationships and identify the declared reference type.
- Check member access and constructor chain.
- Separate compile-time overload resolution from runtime override dispatch.

Quick map / trace / syntax

```
new Student("Asha") -> Person(String) -> Student(String)
```

QUICK RECALL

Trace constructor output in superclass-to-subclass order and check the required super call.

TOPIC 40 / 75 | Inheritance

CODE EXAMPLE AND EXAM HELP

Role of Constructors in Inheritance

Worked code example and output

```
class Person { Person(String name){ System.out.println("Person "+name); } }
class Student extends Person { Student(String n){ super(n); System.out.println("Student");
} }
new Student("Asha");
```

Useful distinction / limitation

If no accessible no-argument superclass constructor exists, the subclass must explicitly call a matching super constructor.

Common mistakes to avoid

- Constructors are not inherited or overridden.
- super(...) must be the first explicit constructor invocation.

Quick viva and follow-up cues

- Core answer: Trace constructor output in superclass-to-subclass order and check the required super call.
- Likely follow-up: If no accessible no-argument superclass constructor exists, the subclass must explicitly call a matching super constructor.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Trace constructor output in superclass-to-subclass order and check the required super call.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 41 / 75 | Inheritance CONCEPT AND DEFINITIONS

Overloading and Polymorphism in Inheritance

Simple definition

Overloading chooses among same-named methods using their parameter lists; overriding lets a subclass provide a replacement behavior selected at runtime.

Formal / technical definition

Overload resolution is primarily compile-time and uses the declared argument types. For an overridden instance method, dynamic dispatch selects the implementation associated with the runtime object's class; fields and static methods are not dynamically overridden in the same way.

Notation and how to read it

overload: same name, different parameters; override: same signature in subtype, compatible return/access rules

How it works

- Draw superclass/subclass relationships and identify the declared reference type.
- Check member access and constructor chain.
- Separate compile-time overload resolution from runtime override dispatch.

Quick map / trace / syntax

```
Animal a = new Dog();
a.sound() -> Dog.sound() at runtime
```

QUICK RECALL

Separate compile-time overload selection from runtime dispatch of an overridden method.

TOPIC 41 / 75 | Inheritance

CODE EXAMPLE AND EXAM HELP

Overloading and Polymorphism in Inheritance

Worked code example and output

```
class Animal { String sound(){ return "?"; } }
class Dog extends Animal { @Override String sound(){ return "bark"; } }
Animal a = new Dog();
System.out.println(a.sound()); // bark
```

Useful distinction / limitation

A child method with a different parameter list is an overload, not an override. Use @Override to catch signature mistakes.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Separate compile-time overload selection from runtime dispatch of an overridden method.
- Likely follow-up: A child method with a different parameter list is an overload, not an override. Use @Override to catch signature mistakes.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Separate compile-time overload selection from runtime dispatch of an overridden method.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 42 / 75 | Package

CONCEPT AND DEFINITIONS

Organizing Classes and Interfaces in Packages

Simple definition

A package groups related classes and interfaces under a name, helping avoid naming collisions and organize a large program.

Formal / technical definition

A package is a namespace for top-level types and package-level access. Package names commonly follow reverse-domain naming conventions; source directory structure generally mirrors the package name for tool compatibility.

Notation and how to read it

package domain.subdomain; -> package path domain/subdomain/

How it works

- Identify the package declaration and fully qualified type name.
- Apply access modifiers using package and subclass location.
- Set classpath roots and platform-specific separators.

Quick map / trace / syntax

```
package name -> namespace + directory convention -> imports/reference by qualified name
```

QUICK RECALL

Show package declaration, source path, and how another class refers to the type.

TOPIC 42 / 75 | Package

CODE EXAMPLE AND EXAM HELP

Organizing Classes and Interfaces in Packages

Worked code example and output

```
package college.records;
public class StudentRecord { }
// file path commonly: college/records/StudentRecord.java
```

Useful distinction / limitation

The package statement, when present, must appear before imports and type declarations in the compilation unit.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Show package declaration, source path, and how another class refers to the type.
- Likely follow-up: The package statement, when present, must appear before imports and type declarations in the compilation unit.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Show package declaration, source path, and how another class refers to the type.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 43 / 75 | Package

CONCEPT AND DEFINITIONS

Package as Access Protection

Simple definition

Package membership can grant access to members with no modifier; protected also allows qualified subclass access under Java's rules.

Formal / technical definition

Access levels are public, protected, package-private, and private. Package-private access is limited to the same package. protected allows same-package access and access from subclasses, with restrictions on cross-package access through references.

Notation and how to read it

private < package-private < protected (with subclass qualification) < public

How it works

- Identify the package declaration and fully qualified type name.
- Apply access modifiers using package and subclass location.
- Set classpath roots and platform-specific separators.

Quick map / trace / syntax

```
same package -> package-private usable
different package non-subclass -> only public API
subclass -> protected rule applies
```

QUICK RECALL

For each access case, name package equality, subclass status, and reference type.

TOPIC 43 / 75 | Package

CODE EXAMPLE AND EXAM HELP

Package as Access Protection

Worked code example and output

```
package p;
class Helper { int value; } // package-private class and member
// another class in package p can use Helper/value; a class in another package cannot use this package-private type
```

Useful distinction / limitation

Protected is not simply 'visible to every subclass through any object'; cross-package access has receiver-type restrictions.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: For each access case, name package equality, subclass status, and reference type.
- Likely follow-up: Protected is not simply 'visible to every subclass through any object'; cross-package access has receiver-type restrictions.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: For each access case, name package equality, subclass status, and reference type.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 44 / 75 | Package

CONCEPT AND DEFINITIONS

Defining Packages

Simple definition

To define a package, put a package declaration at the top of the source file and place the class in the matching folder structure.

Formal / technical definition

A compilation unit may declare at most one package declaration. Public top-level types are normally placed in a correspondingly named source file; a compiler emits class files in package directories relative to the output root.

Notation and how to read it

package college; -> binary name college.Student; output root contains college/Student.class

How it works

- Identify the package declaration and fully qualified type name.
- Apply access modifiers using package and subclass location.
- Set classpath roots and platform-specific separators.

Quick map / trace / syntax

```
source root + package path -> compiled package directory -> qualified class name
```

QUICK RECALL

Distinguish source root, package folder, qualified name, and compiled output root.

TOPIC 44 / 75 | Package

CODE EXAMPLE AND EXAM HELP

Defining Packages

Worked code example and output

```
File: src/college/Student.java
package college;
public class Student { }
Compile from project root: javac -d out src/college/Student.java
```

Useful distinction / limitation

The output directory is a classpath root; do not add the package subfolder itself when resolving college.Student.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Distinguish source root, package folder, qualified name, and compiled output root.
- Likely follow-up: The output directory is a classpath root; do not add the package subfolder itself when resolving college.Student.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Distinguish source root, package folder, qualified name, and compiled output root.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 45 / 75 | Package

CONCEPT AND DEFINITIONS

CLASSPATH Setting for Packages

Simple definition

The class path tells Java tools which directories and JAR files are roots to search for compiled classes and resources.

Formal / technical definition

The class path is a sequence of class-path entries. For a qualified class p.C, the runtime searches for p/C.class beneath each directory/JAR root. Use -cp/-classpath for explicit configuration; path separators differ by operating system.

Notation and how to read it

classpath root + package-relative path -> class file; Windows separator ;, Unix-like separator :

How it works

- Identify the package declaration and fully qualified type name.
- Apply access modifiers using package and subclass location.
- Set classpath roots and platform-specific separators.

Quick map / trace / syntax

```
out
college/Student.class
java -cp out college.Student
```

QUICK RECALL

Identify classpath roots, separator, class-file relative path, and fully qualified main class.

TOPIC 45 / 75 | Package

CODE EXAMPLE AND EXAM HELP

CLASSPATH Setting for Packages

Worked code example and output

```
Compile: javac -d out src/college/Student.java
Run: java -cp out college.Student
Add JAR: java -cp out;lib/tool.jar college.Student (Windows)
Linux/macOS separates entries with :
```

Useful distinction / limitation

Do not put a package name in the main-class argument as a file path; use its fully qualified class name.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Identify classpath roots, separator, class-file relative path, and fully qualified main class.
- Likely follow-up: Do not put a package name in the main-class argument as a file path; use its fully qualified class name.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Identify classpath roots, separator, class-file relative path, and fully qualified main class.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 46 / 75 | Exception Handling

CONCEPT AND DEFINITIONS

The Idea Behind Exceptions

Simple definition

An exception signals an unusual condition during execution and lets code handle or report it separately from the ordinary path.

Formal / technical definition

An exception is an object representing an abnormal condition. Throwing it abruptly completes the current statement sequence and transfers control to the nearest matching handler; without one, it propagates up the call stack.

Notation and how to read it

throw exception -> search current/enclosing handlers -> propagate to caller if unmatched

How it works

- Identify the thrown type and whether it is checked.
- Search for the first compatible handler while unwinding callers.
- Trace catch/finally execution and any propagation.

Quick map / trace / syntax

```
normal path -> statement
exception path -> matching catch / method caller
```

QUICK RECALL

Name the exception, where it is thrown, and which handler can receive it.

TOPIC 46 / 75 | Exception Handling

CODE EXAMPLE AND EXAM HELP

The Idea Behind Exceptions

Worked code example and output

```
try { int n = Integer.parseInt("abc"); }
catch (NumberFormatException ex) { System.out.println("Enter digits"); }
```

Useful distinction / limitation

Exceptions are not a substitute for ordinary branching when a condition is expected and inexpensive to test.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Name the exception, where it is thrown, and which handler can receive it.
- Likely follow-up: Exceptions are not a substitute for ordinary branching when a condition is expected and inexpensive to test.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Name the exception, where it is thrown, and which handler can receive it.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 47 / 75 | Exception Handling

CONCEPT AND DEFINITIONS

Exceptions and Errors: Types

Simple definition

Java throwables include recoverable exception conditions, unchecked programming/runtime failures, and serious VM/system problems called errors.

Formal / technical definition

Throwable is the root of the throwable hierarchy. Exception includes checked exceptions and RuntimeException subclasses (unchecked). Error represents serious conditions generally not intended for application recovery. Both RuntimeException and Error are unchecked.

Notation and how to read it

Throwable -> Exception | Error; RuntimeException extends Exception; Error extends Throwable

How it works

- Identify the thrown type and whether it is checked.
- Search for the first compatible handler while unwinding callers.
- Trace catch/finally execution and any propagation.

Quick map / trace / syntax

```
compile-time checked exception -> catch/declare
unchecked exception/error -> no catch-or-declare requirement
```

QUICK RECALL

Draw the hierarchy and classify one checked exception, one unchecked exception, and one Error.

TOPIC 47 / 75 | Exception Handling

CODE EXAMPLE AND EXAM HELP

Exceptions and Errors: Types

Worked code example and output

```
Throwable
Exception -> IOException (checked), RuntimeException -> NullPointerException (unchecked)
Error -> OutOfMemoryError
```

Useful distinction / limitation

Catching Throwable broadly can hide serious failures. Handle exceptions at a layer that can meaningfully recover or report them.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Draw the hierarchy and classify one checked exception, one unchecked exception, and one Error.
- Likely follow-up: Catching Throwable broadly can hide serious failures. Handle exceptions at a layer that can meaningfully recover or report them.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Draw the hierarchy and classify one checked exception, one unchecked exception, and one Error.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 48 / 75 | Exception Handling CONCEPT AND DEFINITIONS

Control Flow in Exceptions and JVM Reaction

Simple definition

When code throws an exception, normal execution stops and Java searches outward for a compatible catch block.

Formal / technical definition

A throw causes abrupt completion. The JVM searches the current method's exception table and then unwinds callers until a matching handler is found; if none is found, the thread's uncaught-exception mechanism reports it and the thread terminates.

Notation and how to read it

throw point -> same-frame handler -> caller frame -> ... -> uncaught-exception handler

How it works

- Identify the thrown type and whether it is checked.
- Search for the first compatible handler while unwinding callers.
- Trace catch/finally execution and any propagation.

Quick map / trace / syntax

```
try block normal -> continues after catch/finally
throw -> matching catch -> finally -> continuation
```

QUICK RECALL

Trace stack unwinding frame by frame and identify the first compatible handler.

TOPIC 48 / 75 | Exception Handling

CODE EXAMPLE AND EXAM HELP

Control Flow in Exceptions and JVM Reaction

Worked code example and output

```
main -> methodA -> methodB throws IOException
methodB no handler -> methodA handler? if none -> main handler? if none -> uncaught handler
```

Useful distinction / limitation

An uncaught exception normally terminates that thread, not automatically every thread or the entire process.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Trace stack unwinding frame by frame and identify the first compatible handler.
- Likely follow-up: An uncaught exception normally terminates that thread, not automatically every thread or the entire process.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Trace stack unwinding frame by frame and identify the first compatible handler.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 49 / 75 | Exception Handling

CONCEPT AND DEFINITIONS

try, catch, and finally

Simple definition

try marks code that may fail, catch handles a matching exception, and finally is for cleanup that should run as control leaves the construct.

Formal / technical definition

A try statement can have one or more typed catch clauses and an optional finally clause. Catch clauses are checked in order and a superclass catch must not precede a more specific catch. finally runs for normal/exceptional exit in ordinary cases, subject to abrupt JVM termination.

Notation and how to read it

try { ... } catch (SpecificException e) { ... } finally { cleanup; }

How it works

- Identify the thrown type and whether it is checked.
- Search for the first compatible handler while unwinding callers.
- Trace catch/finally execution and any propagation.

Quick map / trace / syntax

```
exception -> catch runs -> finally runs
normal completion -> finally runs
```

QUICK RECALL

Show the exact output/order for normal and exceptional completion, including finally.

TOPIC 49 / 75 | Exception Handling

CODE EXAMPLE AND EXAM HELP

try, catch, and finally

Worked code example and output

```
try { int n = 10 / 0; }
catch (ArithmeticException ex) { System.out.println("cannot divide by zero"); }
finally { System.out.println("done"); }
```

Useful distinction / limitation

A return or throw inside finally can replace an earlier result/exception; avoid control transfer from finally.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Show the exact output/order for normal and exceptional completion, including finally.
- Likely follow-up: A return or throw inside finally can replace an earlier result/exception; avoid control transfer from finally.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Show the exact output/order for normal and exceptional completion, including finally.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 50 / 75 | Exception Handling

CONCEPT AND DEFINITIONS

throw and throws

Simple definition

throw sends one exception object now. throws in a method declaration warns callers that the method may pass specified checked exceptions outward.

Formal / technical definition

The throw statement throws a Throwable-compatible expression and abruptly completes. A throws clause lists exception types a method/constructor may propagate; checked exceptions must be caught or declared unless handled inside.

Notation and how to read it

throw new ExceptionType(...);

ReturnType method(...) throws ExceptionType { ... }

How it works

- Identify the thrown type and whether it is checked.
- Search for the first compatible handler while unwinding callers.
- Trace catch/finally execution and any propagation.

Quick map / trace / syntax

```
throw -> action in method body
throws -> method contract/declaration for callers
```

QUICK RECALL

Contrast the statement keyword with the declaration clause and classify the exception as checked/unchecked.

TOPIC 50 / 75 | Exception Handling

CODE EXAMPLE AND EXAM HELP

throw and throws

Worked code example and output

```
static int parse(String s) throws NumberFormatException {
if (s == null) throw new IllegalArgumentException("missing");
return Integer.parseInt(s);
}
```

Useful distinction / limitation

RuntimeException is unchecked, so it can be thrown without listing it; declaring it may still communicate behavior.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Contrast the statement keyword with the declaration clause and classify the exception as checked/unchecked.
- Likely follow-up: RuntimeException is unchecked, so it can be thrown without listing it; declaring it may still communicate behavior.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Contrast the statement keyword with the declaration clause and classify the exception as checked/unchecked.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 51 / 75 | Exception Handling

CONCEPT AND DEFINITIONS

Inbuilt and User-Defined Exceptions

Simple definition

Java libraries provide exceptions for common failures. A program can define its own exception type for a meaningful domain-specific problem.

Formal / technical definition

Built-in exceptions derive from Throwable subclasses. A custom checked exception typically extends Exception; a custom unchecked exception typically extends RuntimeException. Constructors may pass a message/cause to the superclass.

Notation and how to read it

custom checked -> extends Exception; custom unchecked -> extends RuntimeException

How it works

- Identify the thrown type and whether it is checked.
- Search for the first compatible handler while unwinding callers.
- Trace catch/finally execution and any propagation.

Quick map / trace / syntax

```
domain rule violated -> construct meaningful exception -> caller catches or propagates according to type
```

QUICK RECALL

Choose checked/unchecked base class and give the exception a clear condition and message.

TOPIC 51 / 75 | Exception Handling

CODE EXAMPLE AND EXAM HELP

Inbuilt and User-Defined Exceptions

Worked code example and output

```
class InvalidAgeException extends Exception {
InvalidAgeException(String message){ super(message); }
}
// throw new InvalidAgeException("age must be positive");
```

Useful distinction / limitation

Create a custom type only when it clarifies the failure contract; preserve the original cause when wrapping another failure.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Choose checked/unchecked base class and give the exception a clear condition and message.
- Likely follow-up: Create a custom type only when it clarifies the failure contract; preserve the original cause when wrapping another failure.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Choose checked/unchecked base class and give the exception a clear condition and message.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 52 / 75 | Exception Handling CONCEPT AND DEFINITIONS

Checked and Unchecked Exceptions

Simple definition

Checked exceptions must be caught or declared; unchecked exceptions do not have that compiler requirement.

Formal / technical definition

Checked exception types are Throwable subclasses other than RuntimeException and its subclasses and Error and its subclasses. Unchecked conditions include RuntimeException and Error families. The compiler enforces catch-or-declare for checked exceptions.

Notation and how to read it

checked -> catch or throws; unchecked -> no catch-or-declare rule

How it works

- Identify the thrown type and whether it is checked.
- Search for the first compatible handler while unwinding callers.
- Trace catch/finally execution and any propagation.

Quick map / trace / syntax

```
checked IOException -> compile-time obligation
unchecked NullPointerException -> may occur at runtime without declaration
```

QUICK RECALL

Classify by type hierarchy and state the compiler's obligation for a checked exception.

TOPIC 52 / 75 | Exception Handling

CODE EXAMPLE AND EXAM HELP

Checked and Unchecked Exceptions

Worked code example and output

```
void read() throws IOException { ... } // caller handles/declares
int size = list.get(99); // IndexOutOfBoundsException is unchecked
```

Useful distinction / limitation

Checked does not mean guaranteed to happen; unchecked does not mean harmless or impossible to catch.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Classify by type hierarchy and state the compiler's obligation for a checked exception.
- Likely follow-up: Checked does not mean guaranteed to happen; unchecked does not mean harmless or impossible to catch.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Classify by type hierarchy and state the compiler's obligation for a checked exception.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 53 / 75 | Thread

CONCEPT AND DEFINITIONS

Understanding Threads and Need for Multithreaded Programming

Simple definition

A thread is an independent path of execution inside a process. Multiple threads can keep a program responsive or perform concurrent tasks.

Formal / technical definition

A Java thread is represented by Thread and executes a run method; tasks can also be supplied through Runnable. Threads share process memory while having their own execution stacks, so shared mutable data requires coordination.

Notation and how to read it

Thread object + start() -> JVM schedules run() independently; calling run() directly is an ordinary method call

How it works

- Create a task/thread and call start().
- Identify shared objects and scheduling assumptions.
- Explain benefit and limitation of concurrency.

Quick map / trace / syntax

```
main thread + worker thread share heap objects; each has its own call stack
```

QUICK RECALL

Explain responsiveness/concurrency benefits and distinguish start() from run().

TOPIC 53 / 75 | Thread

CODE EXAMPLE AND EXAM HELP

Understanding Threads and Need for Multithreaded Programming

Worked code example and output

```
Runnable work = () -> System.out.println("task running");
Thread worker = new Thread(work);
worker.start(); // new concurrent execution path
```

Useful distinction / limitation

Use start() to begin a separate thread; calling run() directly does not start a new thread.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Explain responsiveness/concurrency benefits and distinguish start() from run().
- Likely follow-up: Use start() to begin a separate thread; calling run() directly does not start a new thread.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Explain responsiveness/concurrency benefits and distinguish start() from run().

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 54 / 75 | Thread

CONCEPT AND DEFINITIONS

Thread Life-Cycle

Simple definition

A thread moves among states such as new, runnable, blocked/waiting, timed waiting, and terminated as it is started, scheduled, waits, or finishes.

Formal / technical definition

Java Thread.State includes NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, and TERMINATED. RUNNABLE includes execution and readiness-to-run; these API states are not a literal operating-system scheduling diagram.

Notation and how to read it

NEW -> RUNNABLE -> (BLOCKED/WAITING/TIMED_WAITING) -> RUNNABLE -> TERMINATED

How it works

- Start with NEW and identify the operation.
- Map waits, locks, sleeps, or completion to the API state.
- End at RUNNABLE or TERMINATED as appropriate.

Quick map / trace / syntax

```
start() enables scheduling; wait/sleep/join may suspend; completion or uncaught failure ends the thread
```

QUICK RECALL

Use API state names and connect each transition to an operation/event.

TOPIC 54 / 75 | Thread

CODE EXAMPLE AND EXAM HELP

Thread Life-Cycle

Worked code example and output

```
new Thread(task) -> NEW; start() -> RUNNABLE; synchronized lock unavailable -> BLOCKED;
join()/wait() -> WAITING; sleep() -> TIMED_WAITING; run completes -> TERMINATED
```

Useful distinction / limitation

A terminated thread cannot be restarted; calling start() again throws IllegalThreadStateException.

Common mistakes to avoid

- RUNNABLE includes both ready and executing states.
- A terminated Thread instance cannot be restarted.

Quick viva and follow-up cues

- Core answer: Use API state names and connect each transition to an operation/event.
- Likely follow-up: A terminated thread cannot be restarted; calling start() again throws IllegalThreadStateException.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Use API state names and connect each transition to an operation/event.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 55 / 75 | Thread

CONCEPT AND DEFINITIONS

Thread Priorities

Simple definition

Thread priority is a scheduling hint. A higher value does not guarantee that a thread runs first or gets more CPU time.

Formal / technical definition

Java priorities range from Thread.MIN_PRIORITY (1) to MAX_PRIORITY (10), with NORM_PRIORITY (5) as the default. Scheduling policy is implementation/OS dependent; applications should not rely on priority for correctness.

Notation and how to read it

MIN_PRIORITY=1; NORM_PRIORITY=5; MAX_PRIORITY=10

How it works

- Create a task/thread and call start().
- Identify shared objects and scheduling assumptions.
- Explain benefit and limitation of concurrency.

Quick map / trace / syntax

```
priority hint -> scheduler may consider it; no guaranteed order or fairness
```

QUICK RECALL

State the range, default, and why priority does not guarantee ordering.

TOPIC 55 / 75 | Thread

CODE EXAMPLE AND EXAM HELP

Thread Priorities

Worked code example and output

```
worker.setPriority(Thread.NORM_PRIORITY);
worker.start();
```

Useful distinction / limitation

Do not use priority to establish synchronization or deterministic execution order.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: State the range, default, and why priority does not guarantee ordering.
- Likely follow-up: Do not use priority to establish synchronization or deterministic execution order.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: State the range, default, and why priority does not guarantee ordering.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 56 / 75 | Thread

CONCEPT AND DEFINITIONS

Synchronizing Threads

Simple definition

Synchronization protects shared data so that two threads do not perform conflicting updates at the same time.

Formal / technical definition

A synchronized instance method locks its receiver; a synchronized static method locks the Class object. Synchronized blocks acquire a specified monitor. Monitor release/acquisition also establishes memory-visibility ordering.

Notation and how to read it

synchronized method/block -> acquire monitor -> critical section -> release monitor

How it works

- Identify shared state and monitor ownership.
- Trace critical section or wait/notify condition.
- Check memory visibility, rechecking, and deadlock risks.

Quick map / trace / syntax

```
Without lock: read 0 / read 0 / write 1 / write 1 -> lost update
With lock: updates occur one at a time
```

QUICK RECALL

Identify shared state, protecting monitor, critical section, and a possible race without the lock.

TOPIC 56 / 75 | Thread

CODE EXAMPLE AND EXAM HELP

Synchronizing Threads

Worked code example and output

```
class Counter { private int value;
synchronized void increment(){ value++; }
synchronized int get(){ return value; }
}
```

Useful distinction / limitation

Synchronization prevents races on guarded state but can introduce contention or deadlock if locks are acquired carelessly.

Common mistakes to avoid

- Synchronization can protect visibility/atomicity only when shared access follows the same lock discipline.
- Locks can create contention or deadlock.

Quick viva and follow-up cues

- Core answer: Identify shared state, protecting monitor, critical section, and a possible race without the lock.
- Likely follow-up: Synchronization prevents races on guarded state but can introduce contention or deadlock if locks are acquired carelessly.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Identify shared state, protecting monitor, critical section, and a possible race without the lock.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 57 / 75 | Thread

CONCEPT AND DEFINITIONS

Inter-Communication of Threads

Simple definition

Threads can coordinate using wait/notify on a shared object's monitor, typically to wait for a condition such as data becoming available.

Formal / technical definition

Object.wait() releases that object's monitor and suspends until notification/interruption/spurious wakeup/timeout. notify() wakes one waiter; notifyAll() wakes all. Calls require ownership of the object's monitor, and the condition should be checked in a loop after reacquiring it.

Notation and how to read it

while (!condition) wait();; change condition under same monitor; notify/notifyAll

How it works

- Identify shared state and monitor ownership.
- Trace critical section or wait/notify condition.
- Check memory visibility, rechecking, and deadlock risks.

Quick map / trace / syntax

```
wait releases monitor -> thread waits -> notify signals -> waiter reacquires monitor ->
rechecks condition
```

QUICK RECALL

State which object is the monitor and show why the condition is rechecked after waking.

TOPIC 57 / 75 | Thread

CODE EXAMPLE AND EXAM HELP

Inter-Communication of Threads

Worked code example and output

```
synchronized (queue) {
while (queue.isEmpty()) queue.wait();
Object item = queue.remove();
}
// producer adds under same lock, then calls queue.notifyAll();
```

Useful distinction / limitation

Use while, not if, because of spurious wakeups and competition. notify/notifyAll do not release the monitor until the notifier exits its synchronized region.

Common mistakes to avoid

- Call wait/notify while owning the same object's monitor.
- Recheck conditions in a while loop after waking.

Quick viva and follow-up cues

- Core answer: State which object is the monitor and show why the condition is rechecked after waking.
- Likely follow-up: Use while, not if, because of spurious wakeups and competition. notify/notifyAll do not release the monitor until the notifier exits its synchronized region.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: State which object is the monitor and show why the condition is rechecked after waking.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 58 / 75 | Array &String

CONCEPT AND DEFINITIONS

Defining, Initializing, and Accessing an Array

Simple definition

Declare an array reference, create an array object, initialize elements, and access each element by index.

Formal / technical definition

Java array declarations specify a component type. Creation fixes length and initializes elements to default values; an initializer can supply explicit values. Arrays are objects and use zero-based indexing.

Notation and how to read it

Type[] a = new Type[n]; or Type[] a = {value1, value2};; a.length

How it works

- Identify array/string length and valid zero-based indexes.
- Apply the operation and determine whether it mutates or returns a new object.
- State exact resulting value and possible bounds/type errors.

Quick map / trace / syntax

```
names[0] -> Asha; names[1] -> Ravi; names[2] -> invalid
```

QUICK RECALL

Track array length, legal indexes, element type, and default values.

TOPIC 58 / 75 | Array &String

CODE EXAMPLE AND EXAM HELP

Defining, Initializing, and Accessing an Array

Worked code example and output

```
String[] names = new String[2];
names[0] = "Asha";
names[1] = "Ravi";
System.out.println(names[1]); // Ravi
```

Useful distinction / limitation

Reference-type array elements default to null; primitive elements default to zero/false-like values.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Track array length, legal indexes, element type, and default values.
- Likely follow-up: Reference-type array elements default to null; primitive elements default to zero/false-like values.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Track array length, legal indexes, element type, and default values.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 59 / 75 | Array &String

CONCEPT AND DEFINITIONS

Multidimensional Arrays

Simple definition

A multidimensional array is an array whose elements are themselves arrays; rows can even have different lengths.

Formal / technical definition

Java's multidimensional arrays are arrays of arrays. Rectangular arrays use equal row lengths by convention; a jagged array has rows with varying lengths. Each dimension is indexed separately.

Notation and how to read it

Type[][] a; a[row][column]; row lengths: a.length and a[row].length

How it works

- Identify array/string length and valid zero-based indexes.
- Apply the operation and determine whether it mutates or returns a new object.
- State exact resulting value and possible bounds/type errors.

Quick map / trace / syntax

```
grid -> [ [1,2,3], [4,5,6] ]
row 1, column 2 -> 6
```

QUICK RECALL

Trace outer index first, then inner index; check each row's own length.

TOPIC 59 / 75 | Array &String

CODE EXAMPLE AND EXAM HELP

Multidimensional Arrays

Worked code example and output

```
int[][] grid = {{1,2,3},{4,5,6}};
System.out.println(grid[1][2]); // 6
int[][] jagged = new int[2][];
jagged[0] = new int[]{7};
jagged[1] = new int[]{8,9};
```

Useful distinction / limitation

A Java 2D array is not one guaranteed contiguous rectangular object; each row is an array reference.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Trace outer index first, then inner index; check each row's own length.
- Likely follow-up: A Java 2D array is not one guaranteed contiguous rectangular object; each row is an array reference.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Trace outer index first, then inner index; check each row's own length.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 60 / 75 | Array &String

CONCEPT AND DEFINITIONS

Operations on String

Simple definition

String stores text. Common operations inspect, compare, join, split, and extract text; String objects are immutable, so most operations return a new value.

Formal / technical definition

java.lang.String represents sequences of UTF-16 code units and is immutable. Common operations include length, charAt, substring, equals, compareTo, contains, indexOf, replace, trim/strip, and concatenation; API details depend on the Java release.

Notation and how to read it

s.length(); s.charAt(i); s.substring(begin,end); s1.equals(s2); s1 + s2

How it works

- Identify array/string length and valid zero-based indexes.
- Apply the operation and determine whether it mutates or returns a new object.
- State exact resulting value and possible bounds/type errors.

Quick map / trace / syntax

```
String text -> operation -> result String
original object remains unchanged
```

QUICK RECALL

Check zero-based indexes and exclusive substring end; show that an operation does not mutate the original string.

TOPIC 60 / 75 | Array &String

CODE EXAMPLE AND EXAM HELP

Operations on String

Worked code example and output

```
String text = "Java Notes";
System.out.println(text.substring(0,4)); // Java
System.out.println(text.toUpperCase()); // JAVA NOTES
System.out.println(text); // original unchanged
```

Useful distinction / limitation

Use equals for content comparison. For many repeated modifications, StringBuilder avoids creating a long chain of temporary strings.

Common mistakes to avoid

- Strings are immutable; operations return new values.
- Use equals for content and check substring bounds.

Quick viva and follow-up cues

- Core answer: Check zero-based indexes and exclusive substring end; show that an operation does not mutate the original string.
- Likely follow-up: Use equals for content comparison. For many repeated modifications, StringBuilder avoids creating a long chain of temporary strings.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Check zero-based indexes and exclusive substring end; show that an operation does not mutate the original string.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

MODULE-II COMPLETE

Pause. Review. Then continue.

MODULE-II covered inheritance, package organization, exception flow, thread coordination, arrays, and String operations.

Before you begin the next module

- Explain one important rule without looking at the notes.
- Redo a short code trace and identify every output line.
- Use the next roadmap to preview the topics and their connections.

NEXT MODULE STARTS NEXT

MODULE-III - Applet, Event Handling, AWT, Swing, and JDBC

QUICK RECALL

Short regular practice is more useful than memorizing syntax without tracing it.

MODULE-III - what to study and in what order

Module goal

Connect user interaction, GUI components, legacy applet concepts, and database access through JDBC.

Syllabus map - exact topic order

- Applet: Applet and Application; Applet Architecture and Life Cycle; Parameters to Applet.
- Event Handling: Event-Driven Programming in Java; Event Handling Process.
- GUI Programming (Java AWT): Components and Containers; Layout Managers; AWT Components; Adding a Menu to a Window Using Swing Components.
- Introducing Swings: The Origin of Swings; Swing is Built on AWT; Swing Package and Components; Swing Event Handling.
- Database Programming using JDBC: Introduction to JDBC; JDBC Drivers.

Recommended study order

- Treat applet material as a historical syllabus model and learn its lifecycle/parameters.
- Follow event source -> event -> listener for AWT and Swing.
- Finish with JDBC connections, statements, results, and driver choices.

QUICK RECALL

Trace a button event to a listener and a JDBC query to its ResultSet.

TOPIC 61 / 75 | Applet

CONCEPT AND DEFINITIONS

Applet and Application

Simple definition

A Java application starts from a main method under the Java launcher. A historical applet was a small GUI program loaded inside an applet container such as a browser plug-in.

Formal / technical definition

An application is launched by a Java runtime and typically enters through main. An applet extended the applet framework and relied on an AppletContext/container for loading and lifecycle callbacks instead of an ordinary main entry point.

Notation and how to read it

Application -> launcher -> main; applet -> host/container -> lifecycle callbacks

How it works

- Treat the example as the legacy applet model in the syllabus.
- Identify host, lifecycle callback, or parameter flow.
- State current runtime/deployment limitation where relevant.

Quick map / trace / syntax

```
application controls startup through main
legacy applet host controls init/start/stop/destroy
```

QUICK RECALL

Compare startup, host, lifecycle, and current status without implying applets run in modern browsers/JDKs.

TOPIC 61 / 75 | Applet

CODE EXAMPLE AND EXAM HELP

Applet and Application

Worked code example and output

```
Application: public static void main(String[] args) { ... }
Legacy applet: class Demo extends java.applet.Applet { ... }
```

Useful distinction / limitation

Applet browser plug-in deployment was removed in JDK 11; the java.applet API was removed in JDK 26. Applets remain a syllabus/history topic, not a current browser deployment method.

Common mistakes to avoid

- A syllabus applet is historical; modern browser deployment is unavailable.
- Keep applet lifecycle separate from an application's main method.

Quick viva and follow-up cues

- Core answer: Compare startup, host, lifecycle, and current status without implying applets run in modern browsers/JDKs.
- Likely follow-up: Applet browser plug-in deployment was removed in JDK 11; the java.applet API was removed in JDK 26. Applets remain a syllabus/history topic, not a current browser deployment method.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Compare startup, host, lifecycle, and current status without implying applets run in modern browsers/JDKs.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 62 / 75 | Applet

CONCEPT AND DEFINITIONS

Applet Architecture and Life Cycle

Simple definition

An applet container historically created the applet, initialized it, started it when visible, and stopped/destroyed it as the page changed.

Formal / technical definition

The classic applet lifecycle callbacks are init(), start(), paint(Graphics), stop(), and destroy(). The host invokes these callbacks; paint may be requested repeatedly, so persistent setup belongs in init rather than paint.

Notation and how to read it

init: one-time setup; start: active; paint: draw; stop: pause; destroy: final cleanup

How it works

- Treat the example as the legacy applet model in the syllabus.
- Identify host, lifecycle callback, or parameter flow.
- State current runtime/deployment limitation where relevant.

Quick map / trace / syntax

```
container load -> init -> start -> paint
page hidden -> stop
page removed -> destroy
```

QUICK RECALL

Place setup, repeated drawing, pause/resume, and final cleanup in the appropriate lifecycle callback.

TOPIC 62 / 75 | Applet

CODE EXAMPLE AND EXAM HELP

Applet Architecture and Life Cycle

Worked code example and output

```
Legacy lifecycle sketch:
load -> init() once -> start() -> paint() as needed -> stop() -> start() again or destroy()
```

Useful distinction / limitation

This describes the legacy applet model. The API is removed from JDK 26; diagrams are for the syllabus and historical understanding.

Common mistakes to avoid

- paint may run many times; do not perform one-time initialization there.
- Current JDK releases removed the Applet API; diagram the legacy syllabus lifecycle.

Quick viva and follow-up cues

- Core answer: Place setup, repeated drawing, pause/resume, and final cleanup in the appropriate lifecycle callback.
- Likely follow-up: This describes the legacy applet model. The API is removed from JDK 26; diagrams are for the syllabus and historical understanding.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Place setup, repeated drawing, pause/resume, and final cleanup in the appropriate lifecycle callback.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 63 / 75 | Applet

CONCEPT AND DEFINITIONS

Parameters to Applet

Simple definition

A web page historically supplied named text parameters to an applet; the applet read them through the host context.

Formal / technical definition

Classic applet markup used param name/value entries, and the Applet API provided getParameter(String), returning text or null. Numeric/boolean settings required parsing and validation.

Notation and how to read it

name/value text -> host -> getParameter(name) -> String or null

How it works

- Treat the example as the legacy applet model in the syllabus.
- Identify host, lifecycle callback, or parameter flow.
- State current runtime/deployment limitation where relevant.

Quick map / trace / syntax

```
HTML parameter -> applet host -> getParameter("color") -> "blue"
```

QUICK RECALL

Explain the parameter flow and how missing or invalid values should be handled.

TOPIC 63 / 75 | Applet

CODE EXAMPLE AND EXAM HELP

Parameters to Applet

Worked code example and output

```
Legacy example only:
<param name="color" value="blue">
String color = getParameter("color");
// parse/validate before use; API removed in current JDKs
```

Useful distinction / limitation

Parameters are untrusted text: check for null and parse with error handling. Current browser/JDK applet execution is unavailable.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Explain the parameter flow and how missing or invalid values should be handled.
- Likely follow-up: Parameters are untrusted text: check for null and parse with error handling. Current browser/JDK applet execution is unavailable.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Explain the parameter flow and how missing or invalid values should be handled.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 64 / 75 | Event Handling

CONCEPT AND DEFINITIONS

Event-Driven Programming in Java

Simple definition

In event-driven programming, the program waits for actions such as a click or key press, then runs a handler for that event.

Formal / technical definition

An event-driven GUI registers listeners with event sources. When an interaction occurs, the source creates an event object and dispatches it to registered listener methods, typically through a GUI event-dispatch thread.

Notation and how to read it

source + event object + registered listener/handler

How it works

- Name the event source and user action.
- Register the listener and identify the callback.
- Trace the event object and UI response on the event thread.

Quick map / trace / syntax

```
user action -> component/source -> event -> listener method -> UI/model response
```

QUICK RECALL

Name the user action, event source, event object, listener, and resulting action.

TOPIC 64 / 75 | Event Handling

CODE EXAMPLE AND EXAM HELP

Event-Driven Programming in Java

Worked code example and output

A button click creates an ActionEvent; a registered ActionListener handles it and updates a label.

Useful distinction / limitation

Event handlers should remain responsive; long blocking work on the GUI event thread freezes interaction.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Name the user action, event source, event object, listener, and resulting action.
- Likely follow-up: Event handlers should remain responsive; long blocking work on the GUI event thread freezes interaction.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Name the user action, event source, event object, listener, and resulting action.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 65 / 75 | Event Handling CONCEPT AND DEFINITIONS

Event Handling Process

Simple definition

Java GUI code handles an event by creating a component, registering a listener, and implementing the listener's callback.

Formal / technical definition

The delegation event model separates the event source from listener objects. Registration associates a listener with a source; dispatch invokes the listener callback on the appropriate event thread for the toolkit.

Notation and how to read it

component.addXListener(listener);; event -> listener callback(e)

How it works

- Name the event source and user action.
- Register the listener and identify the callback.
- Trace the event object and UI response on the event thread.

Quick map / trace / syntax

```
create source -> implement handler -> register -> user event -> callback runs
```

QUICK RECALL

Trace the registration line separately from the later callback invocation.

TOPIC 65 / 75 | Event Handling

CODE EXAMPLE AND EXAM HELP

Event Handling Process

Worked code example and output

```
button.addActionListener(e -> status.setText("Saved"));
// click -> ActionEvent -> lambda listener -> label text changes
```

Useful distinction / limitation

Registering a listener does not execute it immediately. Swing updates should generally occur on the Event Dispatch Thread.

Common mistakes to avoid

- Listener registration sets up a future callback; it does not call the handler immediately.
- Keep the Swing event thread responsive.

Quick viva and follow-up cues

- Core answer: Trace the registration line separately from the later callback invocation.
- Likely follow-up: Registering a listener does not execute it immediately. Swing updates should generally occur on the Event Dispatch Thread.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Trace the registration line separately from the later callback invocation.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 66 / 75 | GUI Programming (Java AWT) CONCEPT AND DEFINITIONS

Components and Containers

Simple definition

A component is a GUI element such as a button or text field. A container is a component that can hold other components.

Formal / technical definition

AWT Component is the base for visual controls; Container can contain child components. Top-level containers such as Frame host components, while Panel groups controls inside another container.

Notation and how to read it

Component = control/visual element; Container = component that can add child components

How it works

- Choose components and a parent container.
- Select layout and add components in order.
- Register event listeners and display/update the window.

Quick map / trace / syntax

```
Frame -> Panel -> Button / Label / TextField
```

QUICK RECALL

Identify parent container, child components, and the operation that adds each child.

TOPIC 66 / 75 | GUI Programming (Java AWT) CODE EXAMPLE AND EXAM HELP

Components and Containers

Worked code example and output

```
Frame frame = new Frame("Student Form");
Panel panel = new Panel();
Button save = new Button("Save");
panel.add(save); frame.add(panel);
```

Useful distinction / limitation

A container needs an appropriate layout manager to size and position its children.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Identify parent container, child components, and the operation that adds each child.
- Likely follow-up: A container needs an appropriate layout manager to size and position its children.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Identify parent container, child components, and the operation that adds each child.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 67 / 75 | GUI Programming (Java AWT) CONCEPT AND DEFINITIONS

Layout Managers

Simple definition

A layout manager automatically arranges components inside a container as its size changes.

Formal / technical definition

AWT/Swing containers delegate placement and sizing to LayoutManager implementations. Common managers include FlowLayout, BorderLayout, GridLayout, CardLayout, and GridBagLayout.

Notation and how to read it

FlowLayout: row flow; BorderLayout: five regions; GridLayout: equal cells; CardLayout: one visible card; GridBagLayout: flexible grid

How it works

- Choose components and a parent container.
- Select layout and add components in order.
- Register event listeners and display/update the window.

Quick map / trace / syntax

```
Container bounds change -> layout manager recomputes child bounds
```

QUICK RECALL

Choose a layout based on the desired arrangement and describe how it responds to resizing.

TOPIC 67 / 75 | GUI Programming (Java AWT) CODE EXAMPLE AND EXAM HELP

Layout Managers

Worked code example and output

```
Panel p = new Panel(new FlowLayout());
p.add(new Button("One"));
p.add(new Button("Two"));
```

Useful distinction / limitation

Do not assume manual pixel positions adapt well to resizing, localization, or different display sizes.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Choose a layout based on the desired arrangement and describe how it responds to resizing.
- Likely follow-up: Do not assume manual pixel positions adapt well to resizing, localization, or different display sizes.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Choose a layout based on the desired arrangement and describe how it responds to resizing.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 68 / 75 | GUI Programming (Java AWT) CONCEPT AND DEFINITIONS

AWT Components

Simple definition

AWT provides common controls such as labels, buttons, text fields, text areas, checkboxes, choices, lists, and scrollbars.

Formal / technical definition

AWT components are platform-peer-backed GUI classes in java.awt. Components expose properties/events and are placed in containers; heavyweight peer behavior can reflect native platform rendering.

Notation and how to read it

Label: display text; TextField: one-line input; TextArea: multiline; Button: action; Checkbox: on/off; Choice/List: selection

How it works

- Choose components and a parent container.
- Select layout and add components in order.
- Register event listeners and display/update the window.

Quick map / trace / syntax

```
create component -> configure -> add to container -> register listener where needed
```

QUICK RECALL

Name the control, its purpose, its parent container, and any listener it needs.

TOPIC 68 / 75 | GUI Programming (Java AWT) CODE EXAMPLE AND EXAM HELP

AWT Components

Worked code example and output

```
Label label = new Label("Name");
TextField name = new TextField(20);
Checkbox agree = new Checkbox("Agree");
Button submit = new Button("Submit");
```

Useful distinction / limitation

A component only appears when attached to a visible hierarchy and laid out; an event handler is separate from the visual control.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Name the control, its purpose, its parent container, and any listener it needs.
- Likely follow-up: A component only appears when attached to a visible hierarchy and laid out; an event handler is separate from the visual control.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Name the control, its purpose, its parent container, and any listener it needs.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 69 / 75 | GUI Programming (Java AWT) CONCEPT AND DEFINITIONS

Adding a Menu to a Window Using Swing Components

Simple definition

A Swing window can have a menu bar containing menus and menu items; selecting a menu item can trigger an action listener.

Formal / technical definition

JFrame can host a JMenuBar; JMenu groups JMenuItem entries. Menu items participate in the Swing event model and should be created/updated on the Event Dispatch Thread.

Notation and how to read it

JFrame -> JMenuBar -> JMenu -> JMenuItem -> ActionListener

How it works

- Choose components and a parent container.
- Select layout and add components in order.
- Register event listeners and display/update the window.

Quick map / trace / syntax

```
window -> menu bar -> File menu -> Exit item -> click callback
```

QUICK RECALL

Draw the menu hierarchy and identify the component that fires the action event.

TOPIC 69 / 75 | GUI Programming (Java AWT) CODE EXAMPLE AND EXAM HELP

Adding a Menu to a Window Using Swing Components

Worked code example and output

```
JFrame f = new JFrame("Editor");
JMenuBar bar = new JMenuBar();
JMenu file = new JMenu("File");
JMenuItem exit = new JMenuItem("Exit");
file.add(exit); bar.add(file); f.setJMenuBar(bar);
```

Useful distinction / limitation

Set a close operation, size/layout, and visibility; use SwingUtilities.invokeLater for GUI startup in normal applications.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Draw the menu hierarchy and identify the component that fires the action event.
- Likely follow-up: Set a close operation, size/layout, and visibility; use SwingUtilities.invokeLater for GUI startup in normal applications.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Draw the menu hierarchy and identify the component that fires the action event.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 70 / 75 | Introducing Swings

CONCEPT AND DEFINITIONS

The Origin of Swings

Simple definition

Swing expanded Java's GUI toolkit with a richer, more flexible set of controls and a pluggable look and feel.

Formal / technical definition

Swing is a GUI component toolkit introduced as part of the Java Foundation Classes. It supplies components such as JFrame, JButton, JTable, and JTree, with a UI-delegate architecture for look-and-feel variation.

Notation and how to read it

Swing: richer component set + pluggable look and feel + event model built on AWT infrastructure

How it works

- Identify Swing components and their AWT base/container.
- Build the component hierarchy and layout.
- Register listener and keep updates on the Event Dispatch Thread.

Quick map / trace / syntax

```
AWT base/event infrastructure -> Swing component layer -> application GUI
```

QUICK RECALL

State the motivation for Swing and give examples of richer components.

TOPIC 70 / 75 | Introducing Swings

CODE EXAMPLE AND EXAM HELP

The Origin of Swings

Worked code example and output

```
JButton button = new JButton("Submit");
JTable table = new JTable(data, columns);
```

Useful distinction / limitation

Swing is a desktop GUI library; it is not the modern replacement for browser applets or a web UI framework.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: State the motivation for Swing and give examples of richer components.
- Likely follow-up: Swing is a desktop GUI library; it is not the modern replacement for browser applets or a web UI framework.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: State the motivation for Swing and give examples of richer components.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 71 / 75 | Introducing Swings

CONCEPT AND DEFINITIONS

Swing is Built on AWT

Simple definition

Swing uses AWT for lower-level windowing and event infrastructure, while most Swing controls draw and manage their own interface.

Formal / technical definition

Swing classes build on AWT classes and event facilities. Most Swing components are lightweight (do not have an individual native peer), while top-level containers such as JFrame rely on native windows; Swing and AWT components can be mixed with care.

Notation and how to read it

AWT Component/Container + event infrastructure; Swing JComponent subclasses; top-level Swing windows

How it works

- Identify Swing components and their AWT base/container.
- Build the component hierarchy and layout.
- Register listener and keep updates on the Event Dispatch Thread.

Quick map / trace / syntax

```
AWT supplies base GUI abstractions -> Swing adds component/UI layer
```

QUICK RECALL

Explain which parts Swing inherits from AWT and the lightweight/top-level distinction.

TOPIC 71 / 75 | Introducing Swings

CODE EXAMPLE AND EXAM HELP

Swing is Built on AWT

Worked code example and output

```
JButton extends AbstractButton, which is a JComponent (AWT Component descendant); JFrame is a top-level window built on AWT Window/Frame structure.
```

Useful distinction / limitation

Do not say every Swing class is lightweight: Swing top-level containers use native top-level windows.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Explain which parts Swing inherits from AWT and the lightweight/top-level distinction.
- Likely follow-up: Do not say every Swing class is lightweight: Swing top-level containers use native top-level windows.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Explain which parts Swing inherits from AWT and the lightweight/top-level distinction.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 72 / 75 | Introducing Swings

CONCEPT AND DEFINITIONS

Swing Package and Components

Simple definition

Swing controls are mainly in javax.swing. Import the needed classes, place components in a container, and choose a layout.

Formal / technical definition

The Swing API is primarily in the javax.swing package (module java.desktop in modular JDKs). Component classes extend the AWT component hierarchy; common types include JFrame, JPanel, JLabel, JButton, JTextField, and JTable.

Notation and how to read it

javax.swing.*; JFrame top-level; JPanel grouping container; JComponent controls

How it works

- Identify Swing components and their AWT base/container.
- Build the component hierarchy and layout.
- Register listener and keep updates on the Event Dispatch Thread.

Quick map / trace / syntax

```
JFrame -> JPanel/layout -> JLabel + JTextField
```

QUICK RECALL

Name the package, top-level container, intermediate panel, and controls.

TOPIC 72 / 75 | Introducing Swings

CODE EXAMPLE AND EXAM HELP

Swing Package and Components

Worked code example and output

```
import javax.swing.*;
JFrame frame = new JFrame("Demo");
JPanel panel = new JPanel();
panel.add(new JLabel("Name"));
panel.add(new JTextField(12));
frame.setContentPane(panel);
```

Useful distinction / limitation

Swing component naming usually begins with J; AWT classes such as LayoutManager, Color, and Font are still commonly used.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Name the package, top-level container, intermediate panel, and controls.
- Likely follow-up: Swing component naming usually begins with J; AWT classes such as LayoutManager, Color, and Font are still commonly used.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Name the package, top-level container, intermediate panel, and controls.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 73 / 75 | Introducing Swings

CONCEPT AND DEFINITIONS

Swing Event Handling

Simple definition

Swing handles clicks and other actions by registering listeners on components; event callbacks update the interface or application state.

Formal / technical definition

Swing uses the AWT delegation event model. GUI construction and most component updates should occur on the Event Dispatch Thread (EDT); listener work should be brief, with long tasks moved off the EDT and results posted back safely.

Notation and how to read it

EDT -> creates/updates UI and dispatches events; listener registration -> later callback on event thread

How it works

- Identify Swing components and their AWT base/container.
- Build the component hierarchy and layout.
- Register listener and keep updates on the Event Dispatch Thread.

Quick map / trace / syntax

```
EDT: event -> ActionListener -> quick state/UI update -> return to event queue
```

QUICK RECALL

Describe listener registration, event callback, and why the EDT must stay responsive.

TOPIC 73 / 75 | Introducing Swings

CODE EXAMPLE AND EXAM HELP

Swing Event Handling

Worked code example and output

```
SwingUtilities.invokeLater(() -> {
JButton b = new JButton("Save");
b.addActionListener(e -> System.out.println("saved"));
});
```

Useful distinction / limitation

Blocking the EDT makes the GUI unresponsive. Swing components are generally not thread-safe for arbitrary cross-thread updates.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: Describe listener registration, event callback, and why the EDT must stay responsive.
- Likely follow-up: Blocking the EDT makes the GUI unresponsive. Swing components are generally not thread-safe for arbitrary cross-thread updates.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Describe listener registration, event callback, and why the EDT must stay responsive.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 74 / 75 | Database Programming using JDBC CONCEPT AND DEFINITIONS

Introduction to JDBC

Simple definition

JDBC is Java's standard API for connecting Java code to a relational database, sending SQL, and reading results.

Formal / technical definition

JDBC (Java Database Connectivity) defines interfaces and classes in java.sql and javax.sql. A typical flow obtains a Connection, prepares a statement, binds parameters, executes, processes ResultSet, and closes resources.

Notation and how to read it

Java program -> JDBC API -> driver -> database protocol -> SQL result/updates

How it works

- Obtain a driver/connection for the selected database.
- Prepare SQL, bind parameters, execute, and process results.
- Close resources and handle database exceptions.

Quick map / trace / syntax

```
Connection -> PreparedStatement -> executeQuery/executeUpdate -> ResultSet -> close
```

QUICK RECALL

List the JDBC workflow and distinguish executeQuery (rows) from executeUpdate (row-count-changing statements).

TOPIC 74 / 75 | Database Programming using JDBC CODE EXAMPLE AND EXAM HELP

Introduction to JDBC

Worked code example and output

```
try (Connection c = DriverManager.getConnection(url,user,password);
PreparedStatement ps = c.prepareStatement("SELECT name FROM Student WHERE sid=?")) {
ps.setInt(1, 17);
try (ResultSet rs = ps.executeQuery()) { while (rs.next())
System.out.println(rs.getString("name")); }
}
```

Useful distinction / limitation

Use PreparedStatement parameters for values, handle SQLException, and close resources (try-with-resources). Connection URL and driver dependency are database-specific.

Common mistakes to avoid

- Check the declared type/state and the assumptions in the example.
- Trace the Java rule carefully before stating output or behavior.

Quick viva and follow-up cues

- Core answer: List the JDBC workflow and distinguish executeQuery (rows) from executeUpdate (row-count-changing statements).
- Likely follow-up: Use PreparedStatement parameters for values, handle SQLException, and close resources (try-with-resources). Connection URL and driver dependency are database-specific.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: List the JDBC workflow and distinguish executeQuery (rows) from executeUpdate (row-count-changing statements).

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

TOPIC 75 / 75 | Database Programming using JDBC CONCEPT AND DEFINITIONS

JDBC Drivers

Simple definition

A JDBC driver translates JDBC calls into the communication a particular database understands.

Formal / technical definition

Traditional classifications describe Type 1 bridge, Type 2 native-API, Type 3 middleware/network-protocol, and Type 4 pure-Java database-protocol drivers. Modern applications commonly use vendor-provided Type 4 drivers; exact availability and setup are product-specific.

Notation and how to read it

Type 1: bridge; Type 2: native API; Type 3: middleware; Type 4: pure Java direct protocol

How it works

- Obtain a driver/connection for the selected database.
- Prepare SQL, bind parameters, execute, and process results.
- Close resources and handle database exceptions.

Quick map / trace / syntax

```
JDBC interfaces -> driver implementation -> wire protocol -> DB server
```

QUICK RECALL

Compare where translation occurs, native dependencies, and deployment needs for each driver type.

TOPIC 75 / 75 | Database Programming using JDBC CODE EXAMPLE AND EXAM HELP

JDBC Drivers

Worked code example and output

```
Java JDBC code -> vendor driver JAR -> database server; configure the driver's JDBC URL and credentials for the selected DBMS.
```

Useful distinction / limitation

The Type 1-4 taxonomy is a traditional syllabus classification; do not assume every category is current or recommended.

Common mistakes to avoid

- Traditional driver categories describe older deployment styles; Type 4 is the common modern pattern.
- Driver and JDBC URL are specific to the chosen database product.

Quick viva and follow-up cues

- Core answer: Compare where translation occurs, native dependencies, and deployment needs for each driver type.
- Likely follow-up: The Type 1-4 taxonomy is a traditional syllabus classification; do not assume every category is current or recommended.

Exam answer skeleton

- Define the Java construct and state its rule or purpose.
- Trace the code, object/thread/event flow, or result step by step.
- Close with this rule or limitation: Compare where translation occurs, native dependencies, and deployment needs for each driver type.

QUICK RECALL

Predict the result before checking the example; then explain the reason in one sentence.

Rapid revision: MODULE-I definitions (1/2)

Language and OOP

- History, Lineage, and Creation of Java: Java began as a language and platform project for small, connected devices, then grew into a general-purpose platform for desktop, server, and enterprise software.
- Java's Magic Code: The Byte Code: Java source is compiled into bytecode, an intermediate instruction format that a Java Virtual Machine can run on different systems.
- Java Class File Format: A .class file packages bytecode with information the JVM needs to load a class, check it, and connect its methods and fields.
- Java's Buzzwords: Java's well-known design words describe goals such as portability, safety, object-oriented structure, and support for concurrent programs.
- Evolution of Java: Java has changed over time while keeping a strong focus on compatibility; major releases added language features and platform APIs.
- Object-Oriented Concepts: Object-oriented programming organizes a program around objects that combine data with operations on that data.
- Class, Object, and Their Significance: A class describes what its objects contain and can do.
- Environment Variables and Java Setup: Environment variables and command-line options help the operating system and Java tools locate the JDK, compiler, classes, and libraries.

Types and variables

- Java as a Strongly Typed Language: Java checks that values and operations match declared types, so many incompatible uses are caught by the compiler before the program runs.
- Primitive Types: Primitive types hold simple values directly, such as whole numbers, decimal numbers, characters, and true/false values.
- Non-Primitive Types: Non-primitive types describe objects.
- Wrapper Classes: A wrapper class represents a primitive value as an object, which is useful in collections and APIs that work with objects.
- Variables: Scope and Lifetime: Scope is where a variable name can be used.
- Arrays: Declaration and Basic Use: An array stores a fixed number of values of one component type, and each value is reached using an index starting at zero.
- Type Conversion and Casting: A conversion changes a value from one type to another.
- Automatic Type Promotions: In arithmetic, Java may promote smaller numeric values before calculating so the expression uses a wider type.

Rapid revision: MODULE-I definitions (2/2)

Operators and control

- Arithmetic Operators: Arithmetic operators calculate with numeric values: add, subtract, multiply, divide, find a remainder, or change a sign.
- Bitwise Operators: Bitwise operators work on the individual bits of integral values; shifts move bit patterns left or right.
- Relational Operators: Relational operators compare two values and produce a boolean result: true or false.
- Assignment Operators: Assignment stores a value in a variable.
- The ? Operator (Conditional Operator): The conditional operator chooses one of two expressions based on a boolean condition; it is a compact expression form of a two-way choice.
- Operator Precedence: Precedence decides which operator binds first.
- Selection Statements: Selection statements choose which block runs based on a condition or a value.
- Iteration Statements: Iteration statements repeat a block while a condition holds or for each item in a sequence.
- Jump Statements: Jump statements change the normal path through a method or loop: exit a loop, skip an iteration, or return from a method.

Classes and objects

- Class Fundamentals: A class is a blueprint that declares fields for state, methods for behavior, and constructors for initializing new objects.
- Creating and Operating Objects: The new operator creates an object and calls a constructor; a reference then lets the program use its fields and methods.
- Methods and Defining Methods: A method is a named block of code.
- Argument Passing Mechanism: Java always passes arguments by value.
- Method Overloading: Overloading lets one class use the same method name for different parameter lists, so related operations have a familiar name.
- Recursion: Recursion happens when a method solves a problem by calling itself on a smaller case, until a simple base case is reached.
- Nested and Inner Classes: A class may be declared inside another class.
- Anonymous Classes: An anonymous class creates a one-off implementation or subclass right where it is needed, without writing a separate class name.
- Abstract Classes and Interfaces: An abstract class is a partly implemented parent.
- Garbage Collection and Object Reachability: Java reclaims heap memory for objects that can no longer be reached by a running program; the programmer does not manually free each object.
- finalize() Method: finalize() was an old cleanup hook that the garbage collector could call before reclaiming an object, but its timing was unreliable.

Rapid revision: MODULE-II definitions (1/2)

Inheritance and packages

- Use and Benefits of Inheritance in OOP: Inheritance lets a child class reuse and specialize behavior from a parent class, which can reduce duplication when the relationship is truly an is-a relationship.
- Types of Inheritance in Java: Java classes can form single, multilevel, and hierarchical inheritance.
- Inheriting Data Members and Methods: A subclass can use accessible inherited fields and methods, while private superclass members remain hidden from direct subclass access.
- Role of Constructors in Inheritance: A constructor initializes the superclass part first, then the subclass part.
- Overloading and Polymorphism in Inheritance: Overloading chooses among same-named methods using their parameter lists; overriding lets a subclass provide a replacement behavior selected at runtime.
- Organizing Classes and Interfaces in Packages: A package groups related classes and interfaces under a name, helping avoid naming collisions and organize a large program.
- Package as Access Protection: Package membership can grant access to members with no modifier; protected also allows qualified subclass access under Java's rules.
- Defining Packages: To define a package, put a package declaration at the top of the source file and place the class in the matching folder structure.
- CLASSPATH Setting for Packages: The class path tells Java tools which directories and JAR files are roots to search for compiled classes and resources.

Exception handling

- The Idea Behind Exceptions: An exception signals an unusual condition during execution and lets code handle or report it separately from the ordinary path.
- Exceptions and Errors: Types: Java throwables include recoverable exception conditions, unchecked programming/runtime failures, and serious VM/system problems called errors.
- Control Flow in Exceptions and JVM Reaction: When code throws an exception, normal execution stops and Java searches outward for a compatible catch block.
- try, catch, and finally: try marks code that may fail, catch handles a matching exception, and finally is for cleanup that should run as control leaves the construct.
- throw and throws: throw sends one exception object now.
- Inbuilt and User-Defined Exceptions: Java libraries provide exceptions for common failures.
- Checked and Unchecked Exceptions: Checked exceptions must be caught or declared; unchecked exceptions do not have that compiler requirement.

Rapid revision: MODULE-II definitions (2/2)

Threads

- Understanding Threads and Need for Multithreaded Programming: A thread is an independent path of execution inside a process.
- Thread Life-Cycle: A thread moves among states such as new, runnable, blocked/waiting, timed waiting, and terminated as it is started, scheduled, waits, or finishes.
- Thread Priorities: Thread priority is a scheduling hint.
- Synchronizing Threads: Synchronization protects shared data so that two threads do not perform conflicting updates at the same time.
- Inter-Communication of Threads: Threads can coordinate using wait/notify on a shared object's monitor, typically to wait for a condition such as data becoming available.

Arrays and strings

- Defining, Initializing, and Accessing an Array: Declare an array reference, create an array object, initialize elements, and access each element by index.
- Multidimensional Arrays: A multidimensional array is an array whose elements are themselves arrays; rows can even have different lengths.
- Operations on String: String stores text.

Rapid revision: MODULE-III definitions (1/2)

Applets and events

- Applet and Application: A Java application starts from a main method under the Java launcher.
- Applet Architecture and Life Cycle: An applet container historically created the applet, initialized it, started it when visible, and stopped/destroyed it as the page changed.
- Parameters to Applet: A web page historically supplied named text parameters to an applet; the applet read them through the host context.
- Event-Driven Programming in Java: In event-driven programming, the program waits for actions such as a click or key press, then runs a handler for that event.
- Event Handling Process: Java GUI code handles an event by creating a component, registering a listener, and implementing the listener's callback.

AWT GUI

- Components and Containers: A component is a GUI element such as a button or text field.
- Layout Managers: A layout manager automatically arranges components inside a container as its size changes.
- AWT Components: AWT provides common controls such as labels, buttons, text fields, text areas, checkboxes, choices, lists, and scrollbars.
- Adding a Menu to a Window Using Swing Components: A Swing window can have a menu bar containing menus and menu items; selecting a menu item can trigger an action listener.

Rapid revision: MODULE-III definitions (2/2)

Swing

- The Origin of Swings: Swing expanded Java's GUI toolkit with a richer, more flexible set of controls and a pluggable look and feel.
- Swing is Built on AWT: Swing uses AWT for lower-level windowing and event infrastructure, while most Swing controls draw and manage their own interface.
- Swing Package and Components: Swing controls are mainly in javax.swing.
- Swing Event Handling: Swing handles clicks and other actions by registering listeners on components; event callbacks update the interface or application state.

JDBC

- Introduction to JDBC: JDBC is Java's standard API for connecting Java code to a relational database, sending SQL, and reading results.
- JDBC Drivers: A JDBC driver translates JDBC calls into the communication a particular database understands.

Java language, types, and class reference

Compile and run

- javac Hello.java -> Hello.class; java Hello -> launch main.
- JVM loads/verifies class files; bytecode is not native machine code.
- Class file starts with CAFEBABE and includes a version, constant pool, members, and attributes.

Primitive types

- byte, short, int, long; float, double; char; boolean.
- byte/short/char arithmetic normally promotes to int.
- Reference types include class, interface, array, and enum types; a reference can be null.

Operators and control

- Multiplicative -> additive -> shift -> relational -> equality -> bitwise -> logical -> ?: -> assignment.
- if/switch select; while/do/for repeat; break/continue/return transfer control.
- Use parentheses rather than relying on a memorized precedence list for dense code.

Class and method patterns

- class Name { fields; constructors; methods; }
- Type ref = new Type(args); ref.method();
- Overload: distinct parameter lists; override: subclass same signature; dynamic dispatch uses runtime object.
- Java parameters are pass-by-value, including copied reference values.

Inheritance, exceptions, and threads reference

Inheritance and access

- class Child extends Parent implements A,B; one direct superclass, multiple interfaces.
- Constructor chain: superclass first; use super(args) when needed.
- private: declaring class; package-private: same package; protected: package/subclass rules; public: accessible API.

Exception flow

- Throwable -> Exception / Error; RuntimeException is unchecked.
- Checked exception -> catch or declare; throw sends now; throws documents propagation.
- try -> matching catch if thrown -> finally on ordinary exit; unmatched -> caller/uncaught handler.

Thread reminders

- start() begins a thread; direct run() is an ordinary call.
- States: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED.
- synchronized -> monitor; wait releases monitor; notify signals; recheck condition in while.

Arrays and strings

- Array indexes 0..length-1; fixed length; elements have defaults.
- String is immutable; substring(begin,end) excludes end.
- Use equals for String content; use StringBuilder for many repeated changes.

GUI and JDBC workflow reference

Legacy Applet syllabus map

- Applet host historically invoked init/start/paint/stop/destroy.
- getParameter(name) returned supplied text or null.
- Browser plug-in deployment was removed in JDK 11; java.applet API classes were removed in JDK 26.
- Study as legacy architecture; not runnable as a modern browser applet.

AWT / Swing event path

- Container -> layout manager -> components.
- Event source -> event object -> registered listener callback.
- Keep Swing UI creation/updates on the EDT; move long tasks off the event thread.

Menu hierarchy

- JFrame -> JMenuBar -> JMenu -> JMenuItem.
- Add ActionListener; callback responds to selected item.
- Create/display Swing interfaces with SwingUtilities.invokeLater.

JDBC sequence

- Connection -> PreparedStatement -> bind parameters -> execute -> ResultSet/row count -> close.
- executeQuery returns rows; executeUpdate is for DML/DDL-style operations and returns an update count (driver/statement details apply).
- Type 4 driver communicates in the database protocol without a native client library.

Frequently asked exam and viva themes

MODULE-I - language and types

- History, Java design goals, bytecode, and class-file structure.
- Primitive/reference values, wrapper classes, scope, casts, and promotions.
- Operators, precedence, selection, loops, and jump statements.

MODULE-I - classes

- Class/object, constructors, methods, overloads, recursion, and parameter passing.
- Nested/anonymous types and abstract classes/interfaces.
- Reachability, garbage collection, and finalize() status.

MODULE-II - inheritance and runtime

- Inheritance benefits/types, access, constructor sequence, and dynamic dispatch.
- Packages and classpath; exception types and flow.
- Thread states, race conditions, synchronization, and wait/notify.

MODULE-III - UI and database

- Applet lifecycle/parameters as a legacy syllabus topic.
- AWT/Swing hierarchy, layout, events, listeners, and EDT.
- JDBC connections, statements, results, and driver types.

How to earn method marks

- Write the rule before the result.
- Show type, reference, state, call stack, or event flow.
- Trace intermediate values and output.
- Name legacy/current API status where relevant.

Viva response pattern

- Give one clear definition.
- Name the syntax or diagram.
- Use one small example and trace it.
- Finish with a limitation or common error.

Module 1 exam practice and answer cues

Frequently asked long-answer questions

- Explain Java's lineage, compilation to bytecode, and the role of JVM/class files.
- Describe Java's type system, primitive/reference types, wrapper classes, and casting.
- Trace automatic promotions and operator precedence in a mixed expression.
- Compare selection, iteration, and jump statements with code traces.
- Explain class/object, constructors, methods, overloading, and argument passing.
- Compare nested/inner/anonymous classes, abstract classes, and interfaces.
- Explain garbage collection, reachability, and why finalize() should not be used.

Short-answer / viva

- What is bytecode?
- What does CAFEBABE identify?
- Are Java strings primitive?
- What does a wrapper do?
- Is Java pass-by-reference?
- What does a constructor return?
- Does System.gc() force collection?

Easy

- Name all eight primitive types.
- Find the output of 17/5 and 17%5 for int operands.
- Distinguish a class, object, and reference.
- Trace a for loop that sums 1 through 3.

Moderate

- Trace a byte + byte expression and state its type.
- Compare == and equals for two equal-content String objects.
- Show caller-visible effects of mutating an array parameter.
- Trace one recursive factorial call sequence.

Hard

- Explain overload resolution versus runtime overriding using a superclass reference.
- Trace constructor calls and initialization order across a two-class hierarchy.
- Compare abstract class/interface choices for a shared API and state data/behavior trade-offs.

Answer cues / final checks

- Java is statically and strongly typed; some failures remain runtime errors.
- Byte/short/char arithmetic normally promotes to int.
- Java passes all arguments by value, including reference values.
- Overloading is selected at compile time; overrides dispatch by runtime class.
- GC is nondeterministic; finalization is deprecated for removal.

Module 2 exam practice and answer cues

Frequently asked long-answer questions

- Explain inheritance benefits, Java inheritance shapes, member access, and constructor chaining.
- Compare overloading and overriding and trace polymorphic dispatch.
- Explain package naming, access protection, and classpath roots.
- Trace exception control flow and compare checked/unchecked exceptions and Errors.
- Explain thread lifecycle, priorities, synchronization, and wait/notify communication.
- Trace array initialization, multidimensional indexing, and String operations.

Short-answer / viva

- Can a Java class extend two classes?
- Are constructors inherited?
- What is a package-private member?
- What happens to an uncaught exception?
- Why use while with wait()?
- Is String mutable?

Easy

- Give examples of single, multilevel, and hierarchical inheritance.
- List all six values of Java Thread.State.
- Write a package declaration and a fully qualified class name.
- Print an array element using a valid index.

Moderate

- Trace which override executes through a superclass reference.
- Show superclass and subclass constructor output order.
- Classify IOException, NullPointerException, and OutOfMemoryError.
- Explain why wait releases the monitor and notify alone does not release it.

Hard

- Trace exceptions across three method frames and locate the selected catch/finally path.
- Explain a race condition on a shared counter and show a monitor-based fix.
- Compare classpath root with package directory for launching a packaged class.

Answer cues / final checks

- A Java class extends one class but may implement multiple interfaces.
- Constructors initialize superclass before subclass and are not inherited.
- RUNNABLE covers executing and ready-to-run states.
- wait must occur while holding the target object's monitor; loop to retest condition.
- A String method returns a new value; the original String is unchanged.

Module 3 exam practice and answer cues

Frequently asked long-answer questions

- Compare a Java application with a legacy applet and explain the applet lifecycle/parameters.
- Explain event-driven programming and the source-event-listener process.
- Describe AWT components, containers, and layout managers.
- Build a Swing menu hierarchy and explain action handling on the EDT.
- Explain Swing's relationship with AWT and common Swing packages/components.
- Describe a JDBC program's connection, statement, execution, result, and cleanup flow.
- Compare the four traditional JDBC driver categories.

Short-answer / viva

- Who calls an applet lifecycle method?
- What does getParameter return?
- What is an event source?
- Name three layout managers.
- What is the Swing EDT?
- Which JDBC object represents query rows?

Easy

- State one difference between applet and application entry points.
- Draw Frame -> Panel -> Button.
- Name the child controls of a simple Swing menu.
- List the JDBC classes in query order.

Moderate

- Trace a button click to its ActionListener callback.
- Choose BorderLayout vs GridLayout for two different interface needs.
- Explain why database-specific driver JAR/URL are required.
- State why an applet example is legacy-only in current Java.

Hard

- Design a simple Swing form and describe EDT, layout, and listener responsibilities.
- Trace a JDBC query with PreparedStatement parameters and ResultSet iteration.
- Compare driver types by where translation occurs and native dependency.

Answer cues / final checks

- Legacy applet lifecycle: init, start, paint, stop, destroy.
- An event source dispatches an event object to registered listeners.
- Swing UI tasks belong on the Event Dispatch Thread.
- JDBC flow: Connection -> PreparedStatement -> ResultSet -> close.
- Type 4 drivers are pure Java and speak the database protocol directly.

Additional / Outside Syllabus: modern Java habits

Habits that help beyond exam snippets

- Use meaningful names, small methods, and clear access boundaries.
- Prefer interfaces for stable behavior contracts and composition for has-a relationships.
- Close files, sockets, and database resources deterministically with try-with-resources.
- Keep shared mutable state small; synchronize consistently or use concurrency utilities appropriate to the task.
- These are general development habits and are not extra syllabus headings.

Modern status notes relevant to legacy syllabus topics

- Applets remain here for the syllabus/history; browser plug-in deployment was removed in JDK 11 and the java.applet API was removed in JDK 26.
- Object.finalize() remains deprecated for removal as of JDK 27; do not build new cleanup logic around it.
- Current release/version details were checked on 7 October 2026; the syllabus remains the source of topic organization.

QUICK RECALL

A clear trace is better than memorizing output without understanding object state.

Additional / Outside Syllabus: project and debugging checklist

Before compiling a small project

- Check that the public class name matches its source filename.
- Check package declaration, folder path, and classpath root.
- Compile with the intended JDK and run the fully qualified main class.
- For GUI, keep event callbacks short and create Swing UI on the EDT.
- For JDBC, verify driver JAR, URL, credentials, and resource closure.

Debug with a trace

- Start at the first unexpected value or exception, not at the final output alone.
- For object code, write declared type and runtime object class separately.
- For exceptions, walk outward through the call stack to the first matching catch.
- For threads and GUI events, identify which thread owns the current action.

QUICK RECALL

Find the earliest point where the actual execution differs from the intended trace.

Sources and current-context notes

Current status checked 7 October 2026

- Oracle lists JDK 27 as the current Java SE release; Java SE 27 GA was published on 15 September 2026.
- Browser plug-in deployment for applets was removed in JDK 11; the java.applet API classes were removed in JDK 26. The syllabus applet material is therefore taught as a legacy model.
- Finalization is deprecated for removal in current Java documentation. The notes explain finalize() for the syllabus and advise deterministic resource cleanup.
- Release information is time-sensitive; check the Oracle release notes when installing or targeting a specific JDK.

Primary references

- Oracle, The Java Language Environment (history/design goals): https://www.oracle.com/java/technologies/language-environment.html
- Oracle, Java SE 27 Language Changes Summary: https://docs.oracle.com/en/java/javase/27/language/java-language-changes-summary.html
- Oracle, Java SE 27 JVM Specification, class-file format: https://docs.oracle.com/javase/specs/jvms/se27/html/jvms-4.html
- Oracle, APIs Removed in JDK 26 (includes java.applet): https://docs.oracle.com/en/java/javase/26/migrate/removed-apis.html
- Oracle, Java SE 27 Deprecated API List (finalization): https://docs.oracle.com/en/java/javase/27/docs/api/deprecated-list.html
- Oracle, Consolidated JDK 27 Release Notes: https://www.oracle.com/java/technologies/javase/27all-relnotes.html
- Java SE specifications and API documentation: https://docs.oracle.com/javase/specs/

Created by Anurag Yadav

Email: annuanuragyadav908@gmail.com